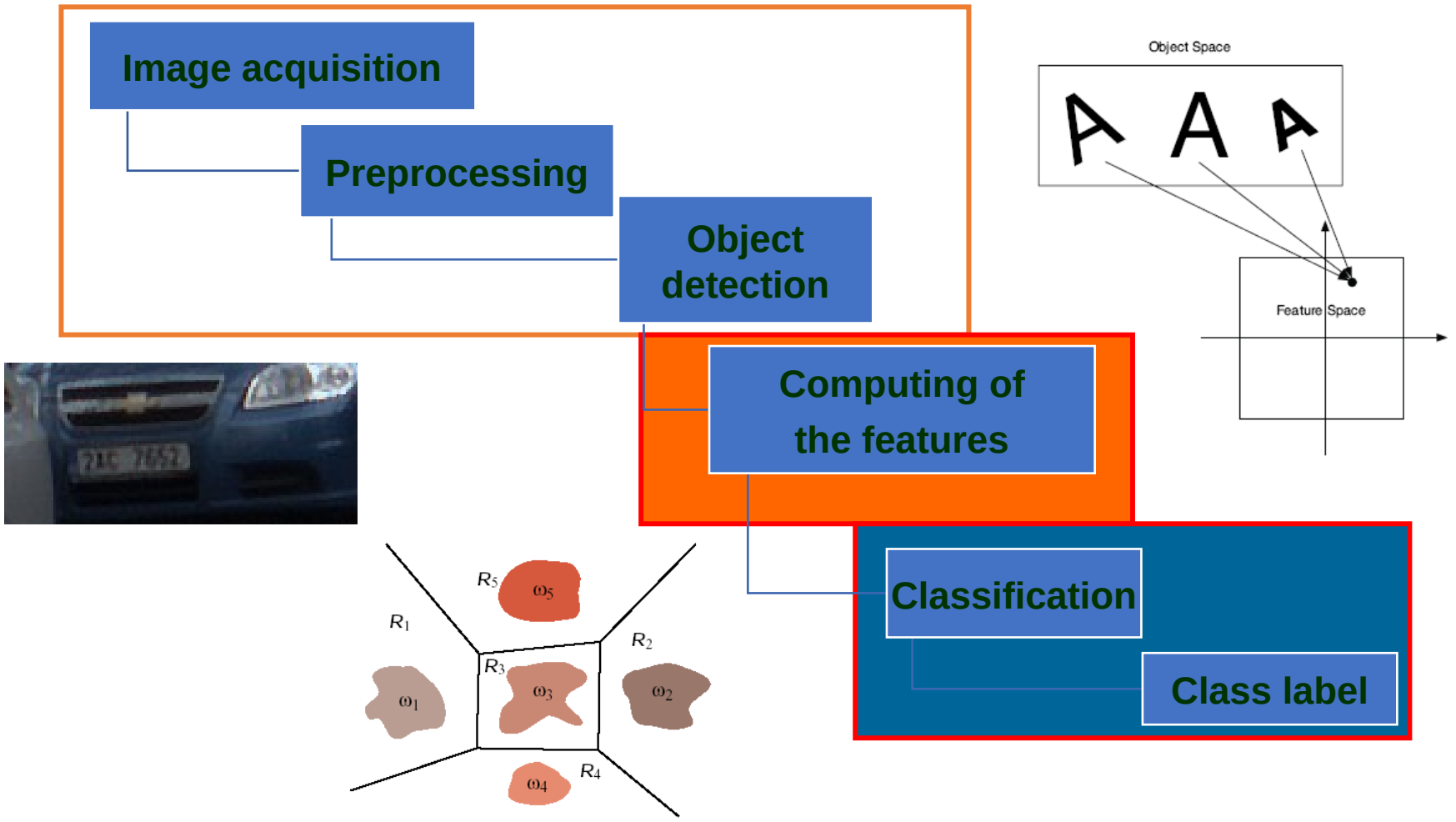
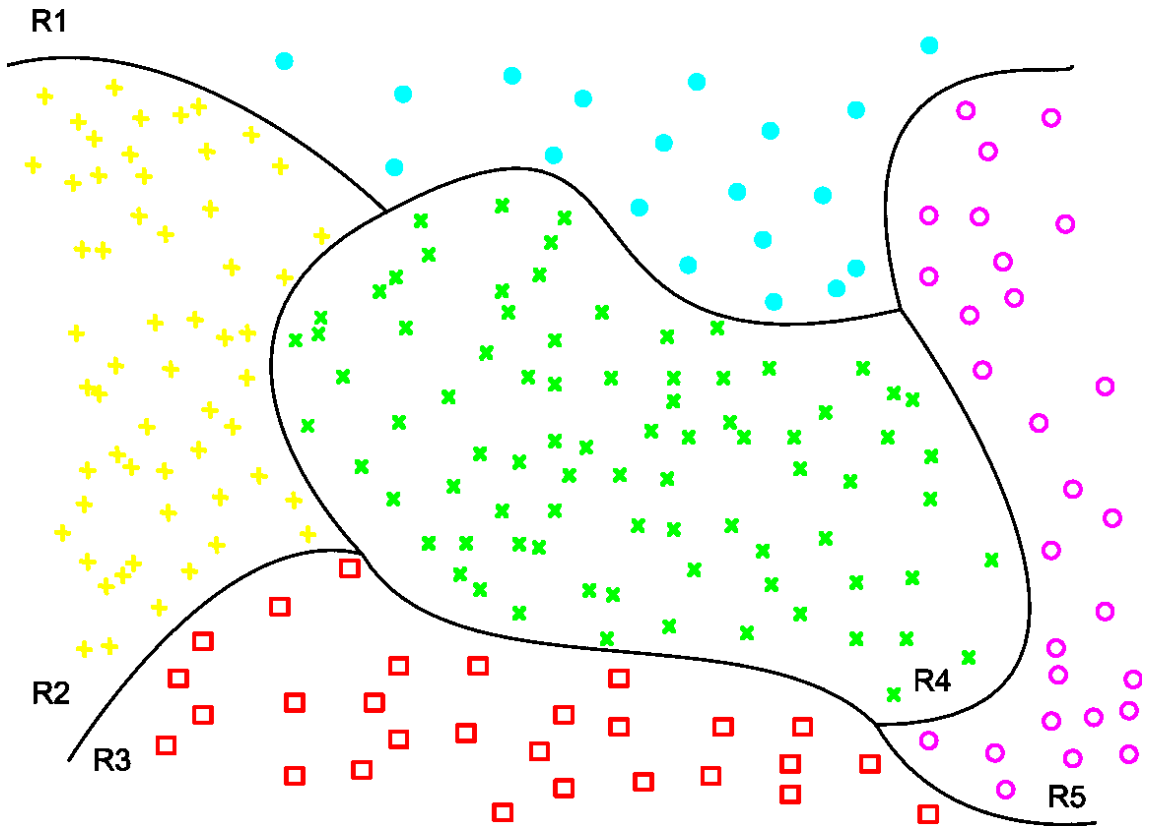
Visual Object Recognition



Classification rule setup

Equivalent to a partitioning of the feature space

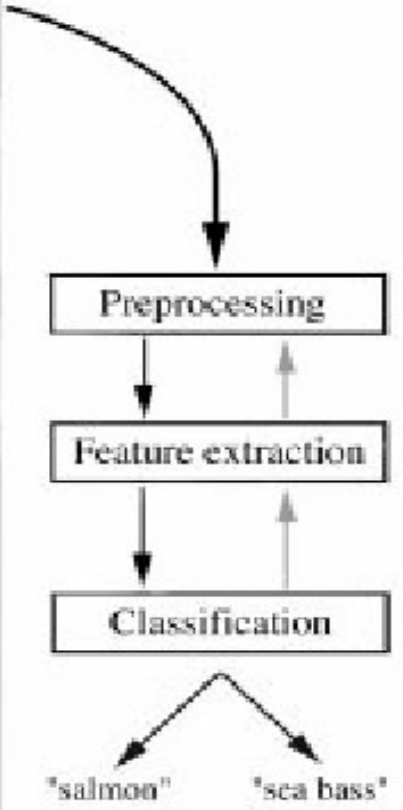


Independent of the particular application

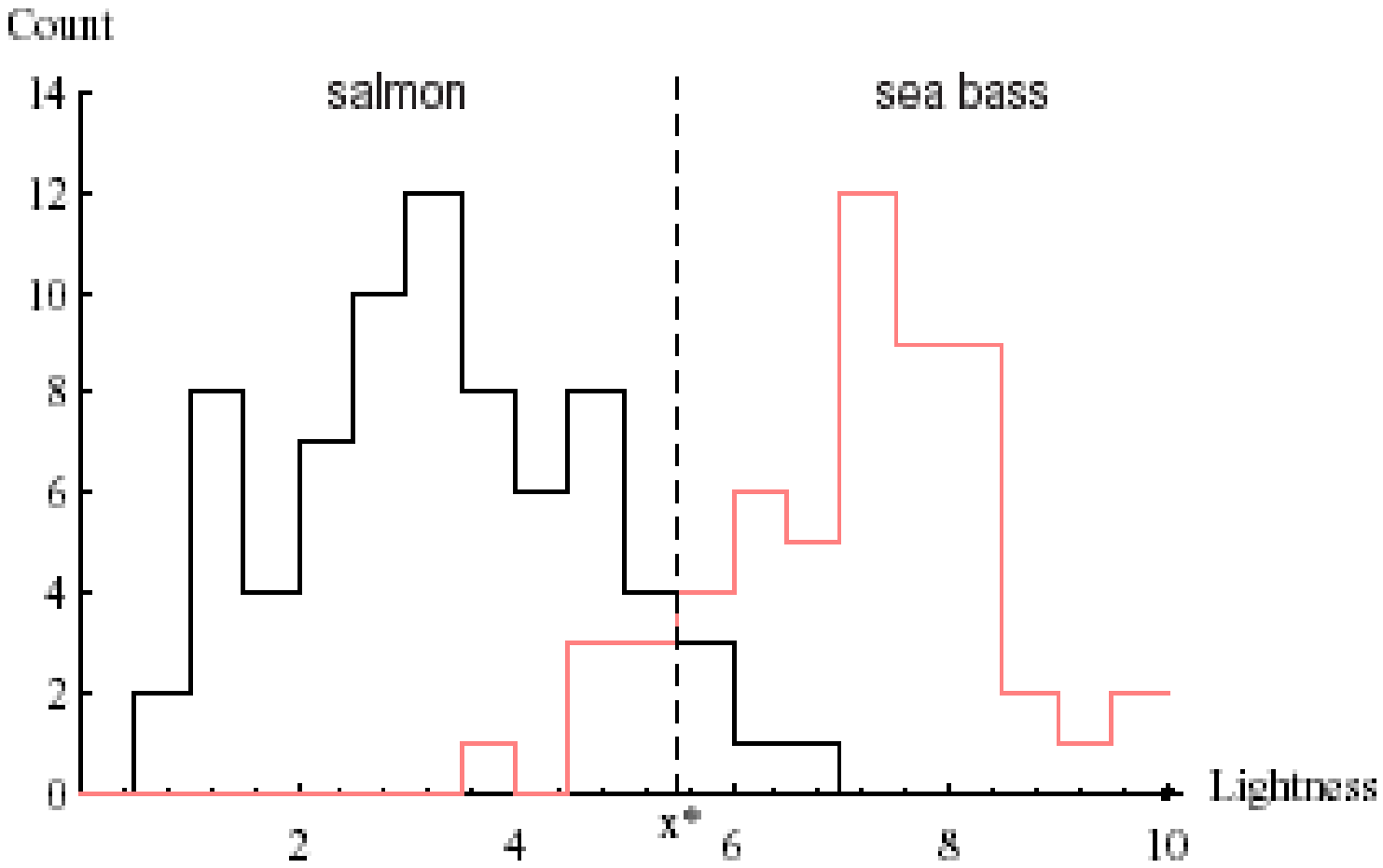
Pattern Recognition

- **Supervised PR**
  - **– training set available for each class**
- **Unsupervised PR (clustering)**
  - **– training set not available, No. of classes may not be known**

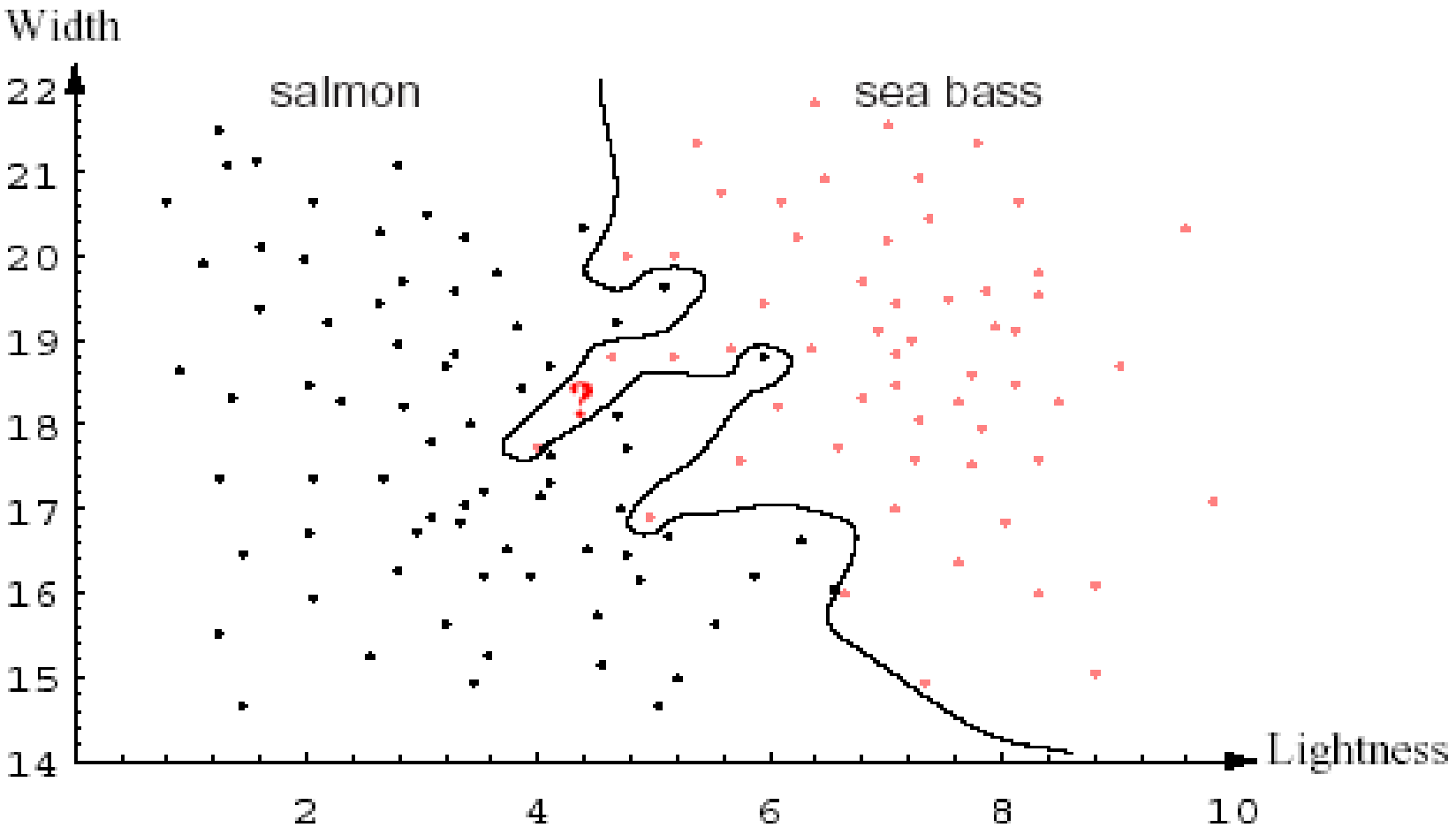
An example – Fish classification



The features: Length, width, brightness



2-D feature space



Empirical observation

- For a given training set, we can have several classifiers (several partitioning of the feature space)

Empirical observation

- For a given training set, we can have several classifiers (several partitioning of the feature space)

- The training samples are not always classified correctly

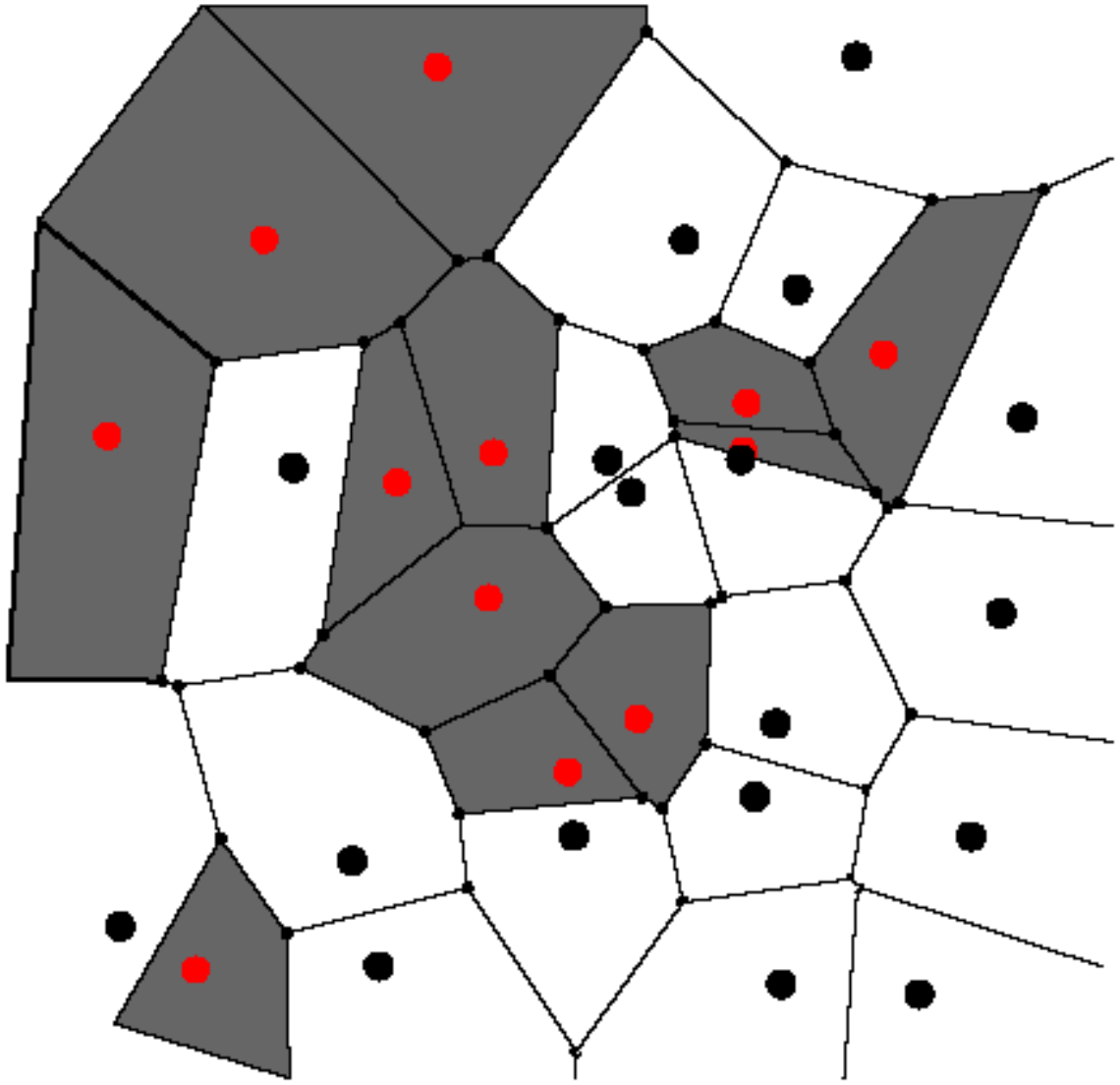
Empirical observation

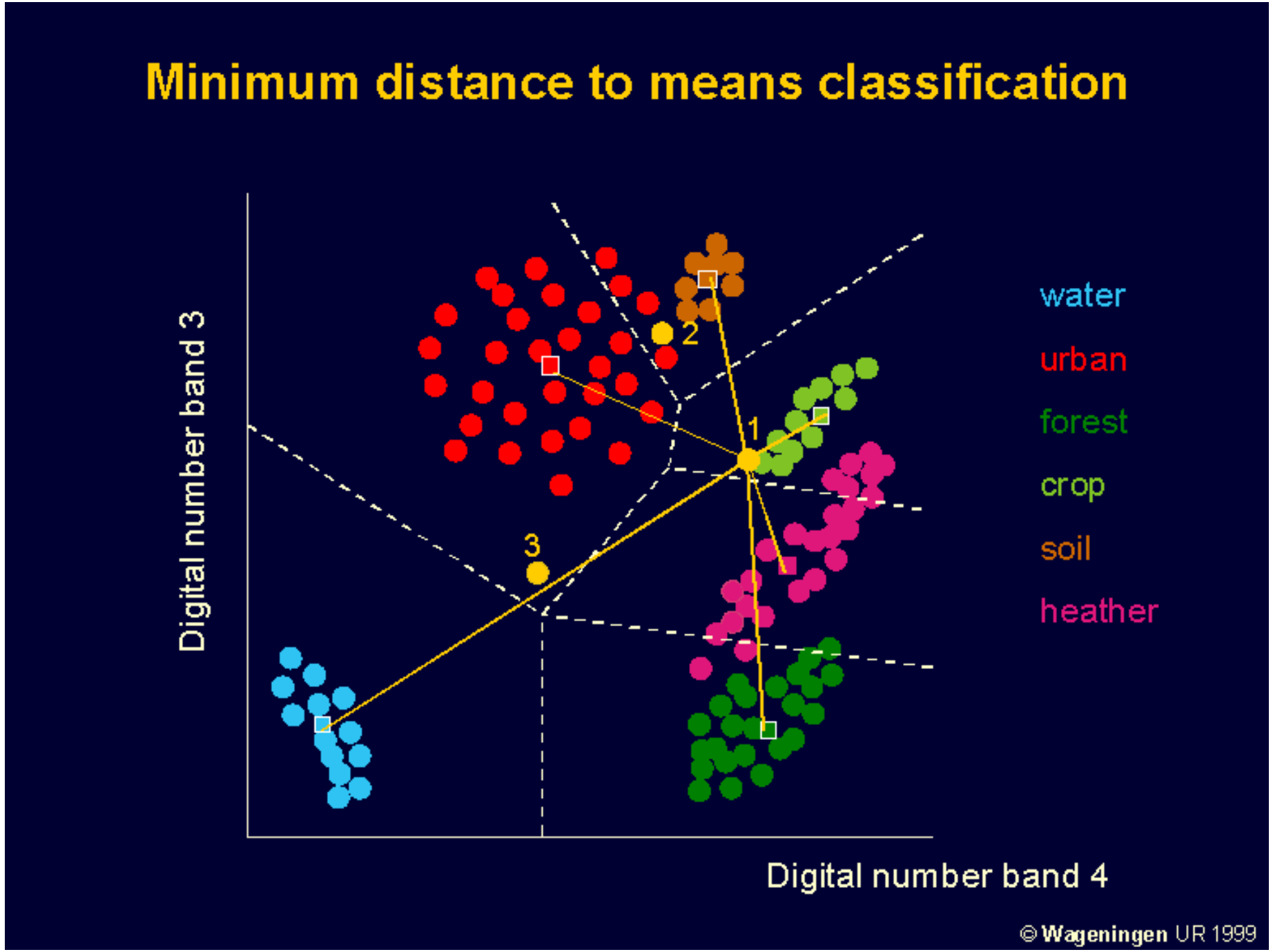
- For a given training set, we can have several classifiers (several partitioning of the feature space)
- The training samples are not always classified correctly
- We should avoid overtraining of the classifier

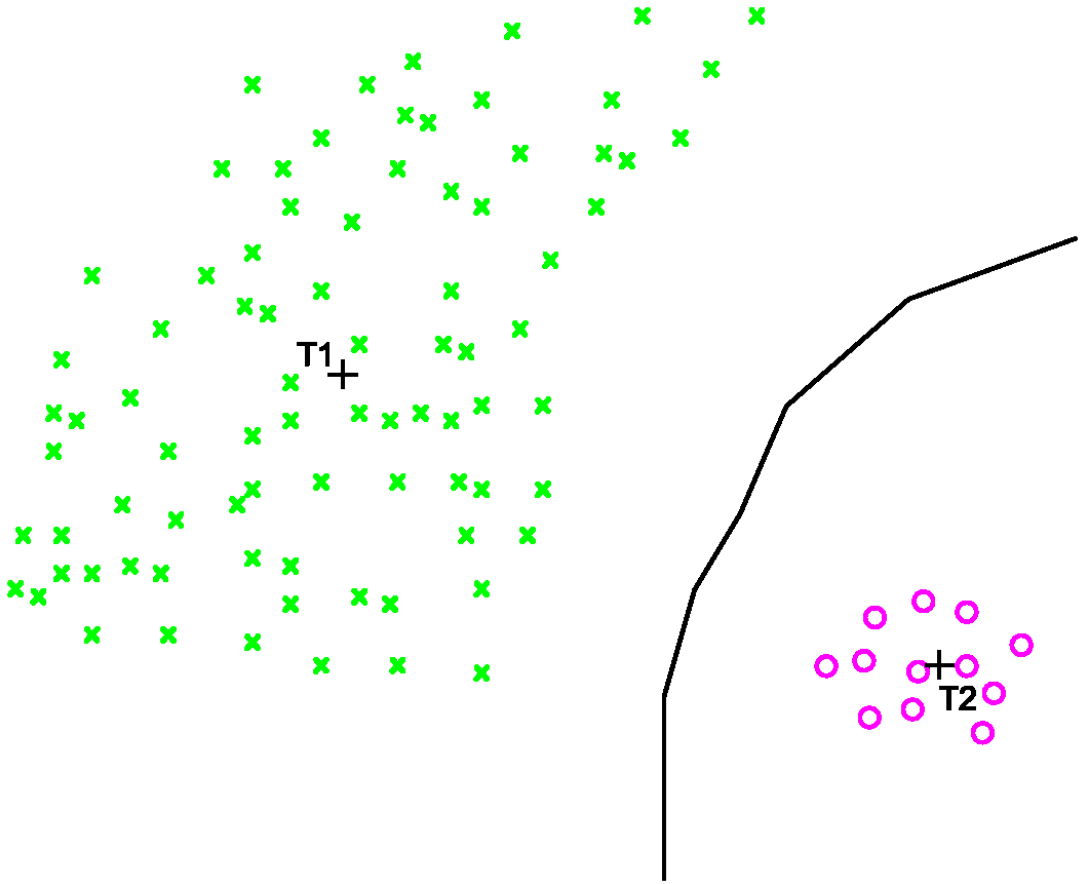
Minimum distance (NN) classifier

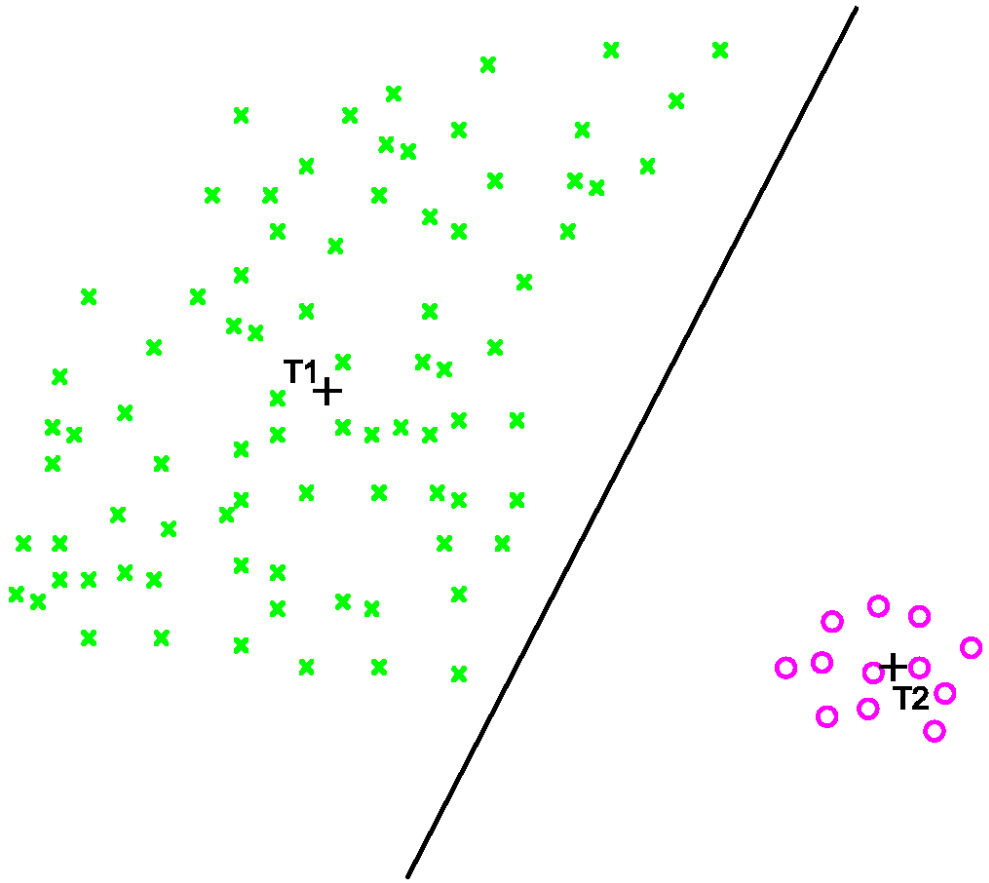
- Depending on $d(\mathbf{x}, \omega_i)$ NN classifier may not be linear
- NN classifier is sensitive to outliers ✉ k-NN classifier

Voronoi polygons




Minimum distance to means classification
Digital number band 3
Digital number band 4
water
urban
forest
crop
soil
heather
1
2
3
© Wageningen UR 1999

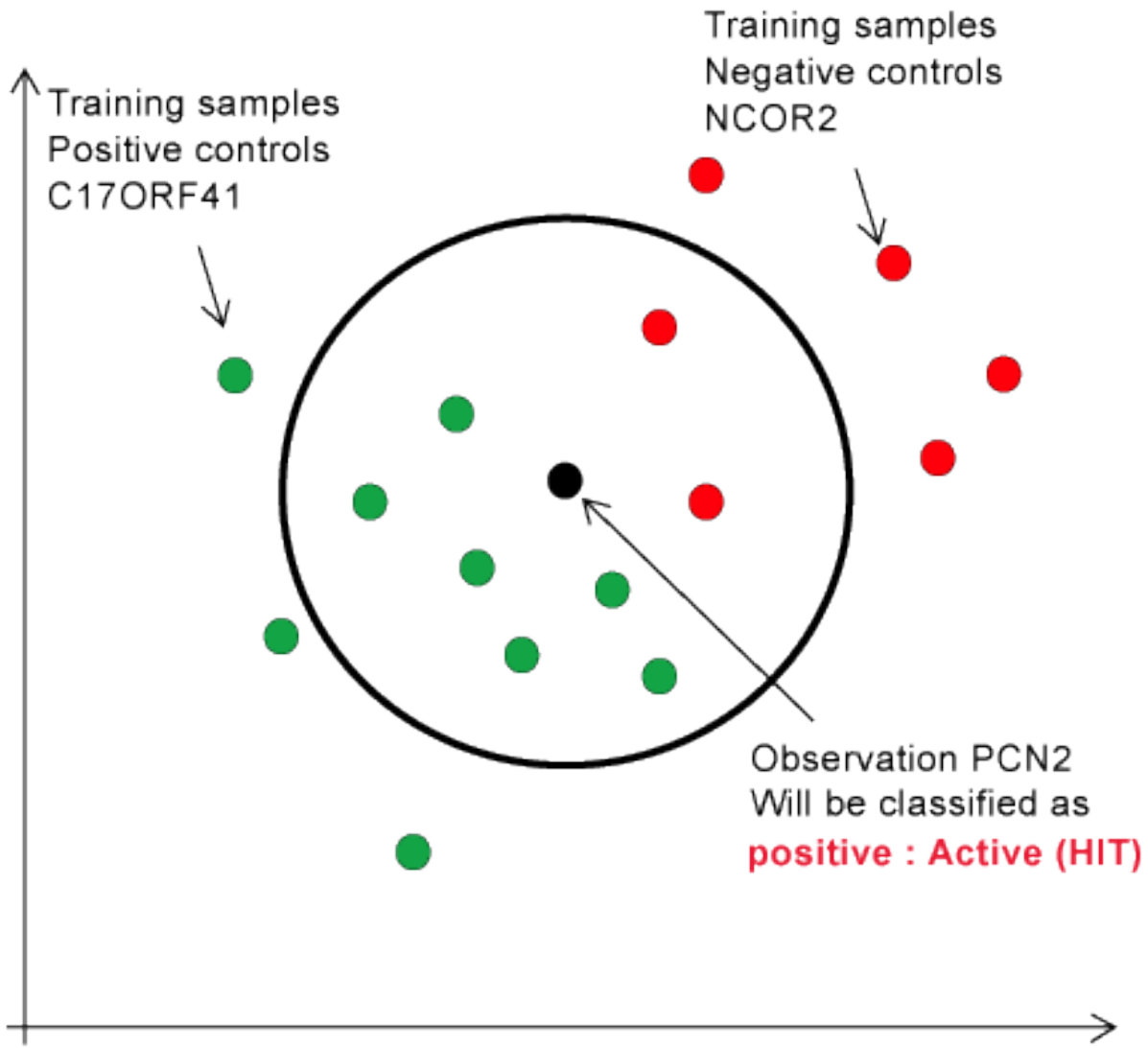

T1
T2


T1
T2

k-NN classifier

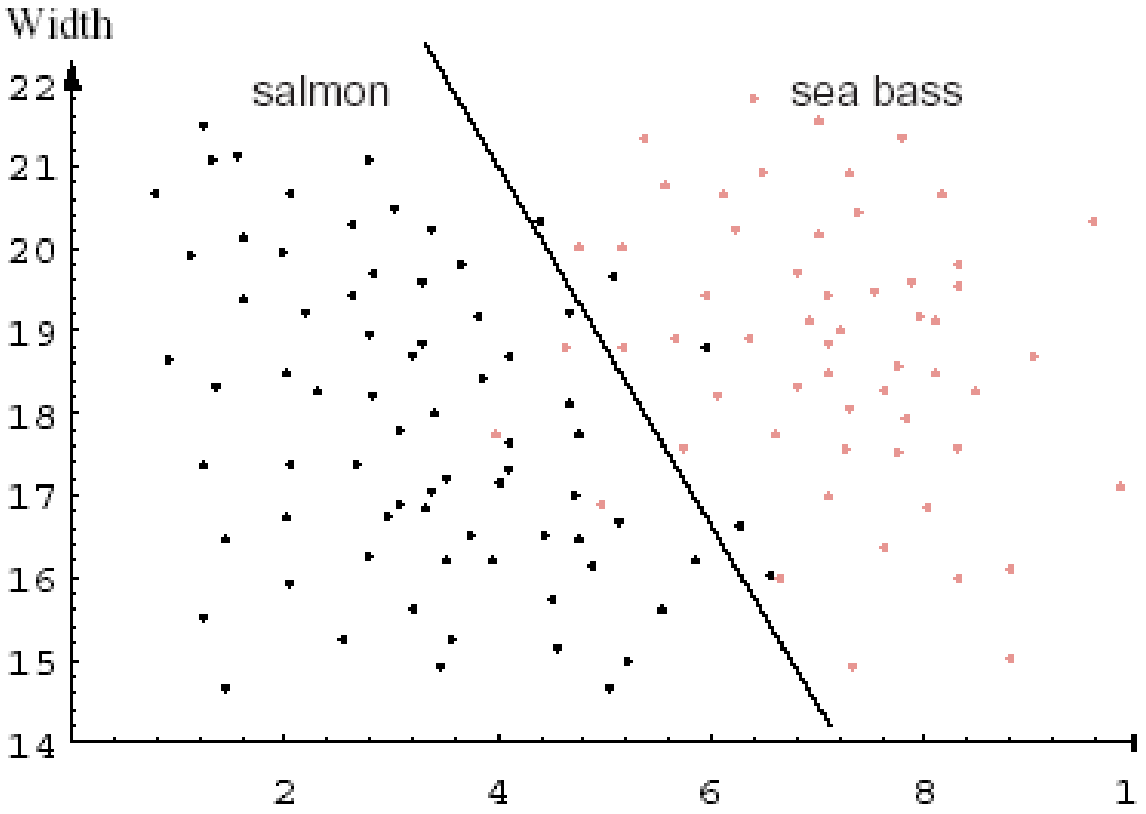
- NN classifier is sensitive to outliers ✉ k-NN classifier
- It finds the nearest training points unless *k* samples belonging to one class has been reached

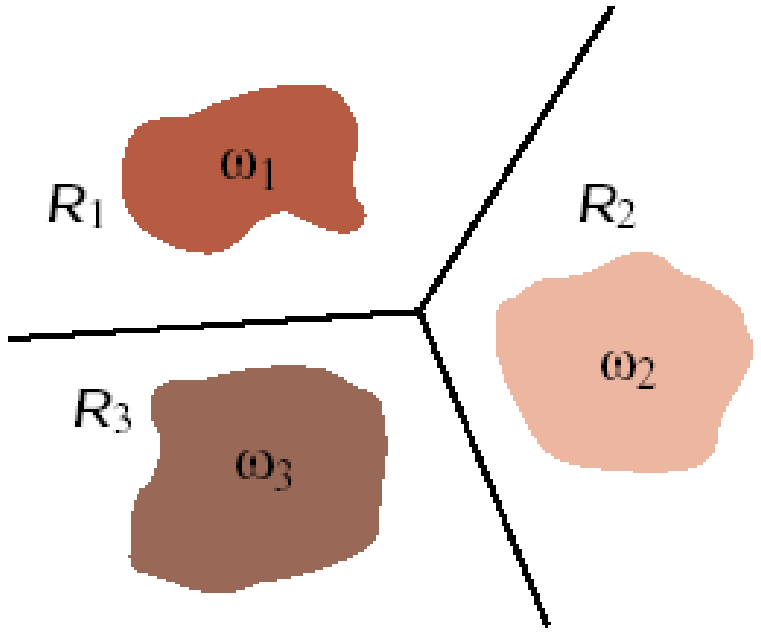
k-NN classifier



Linear classifier

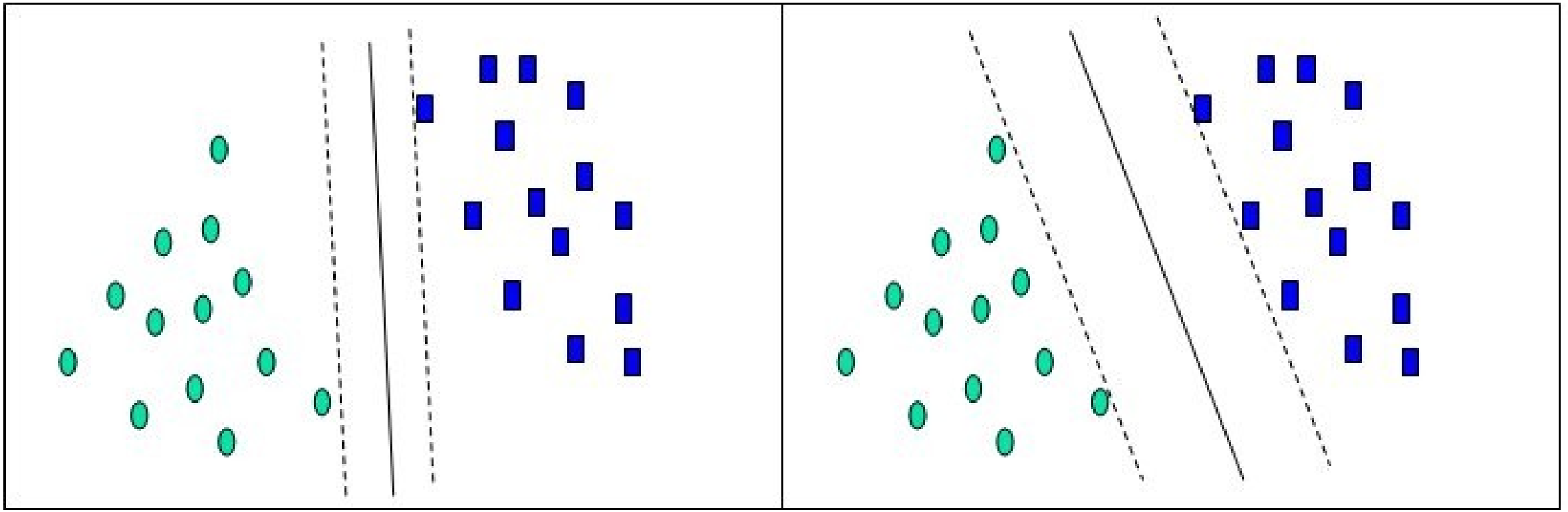
Discriminant functions g(x) are hyperplanes





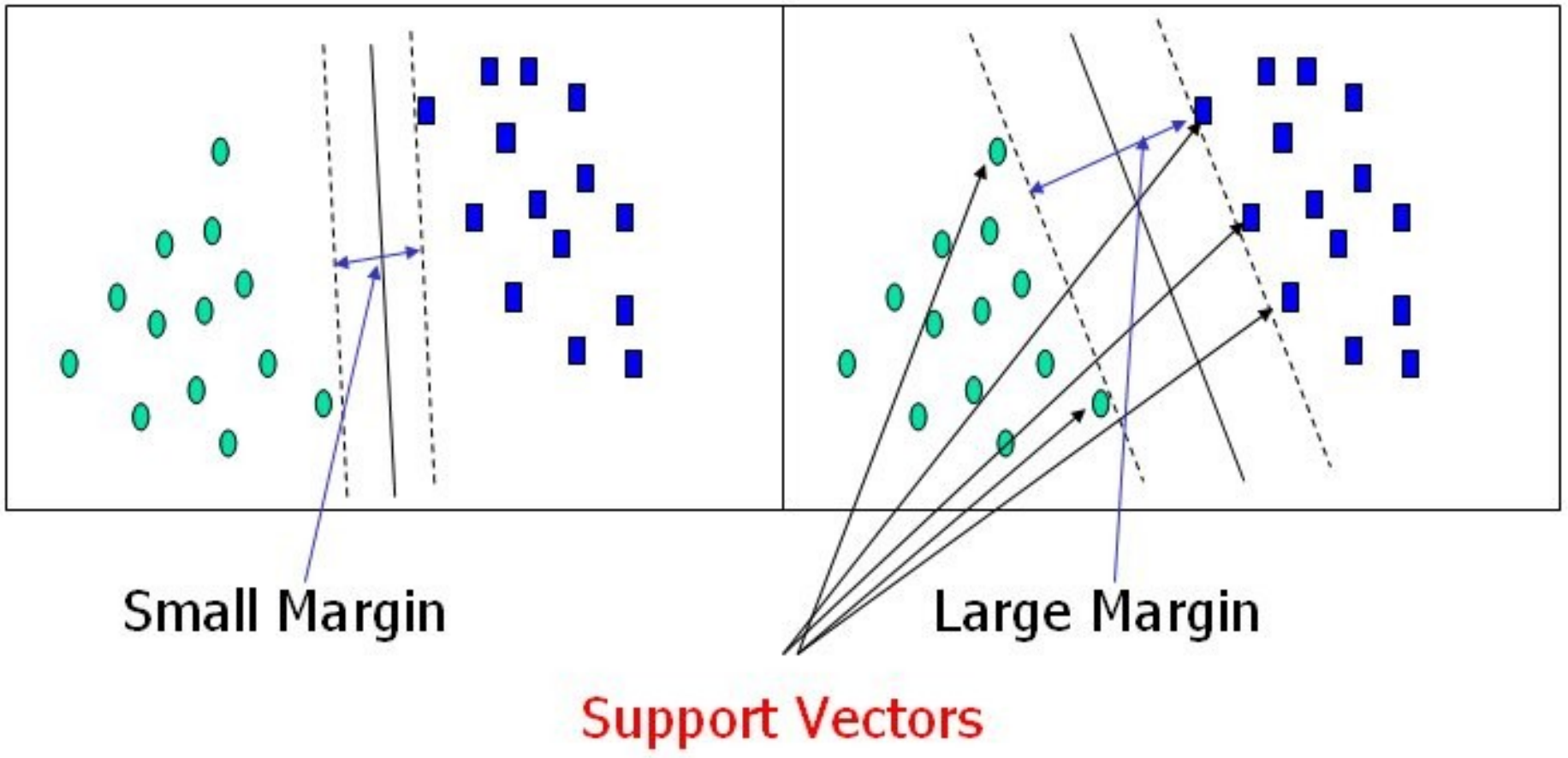
„Optimal“ linear classifier

Maximizing the margin



Support vector machine (SVM)

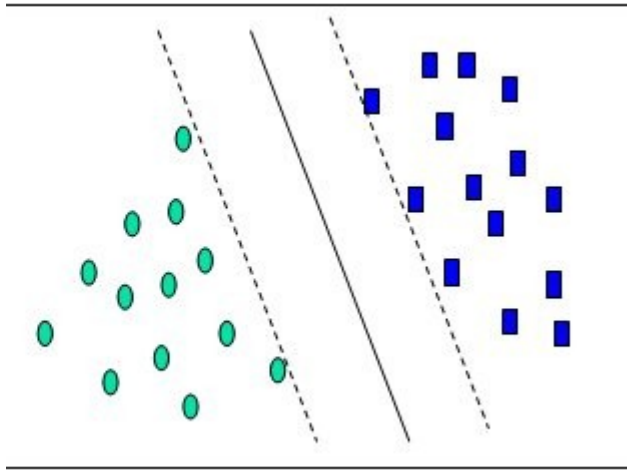
“Optimal” linear classifier



Support vector machine (SVM)

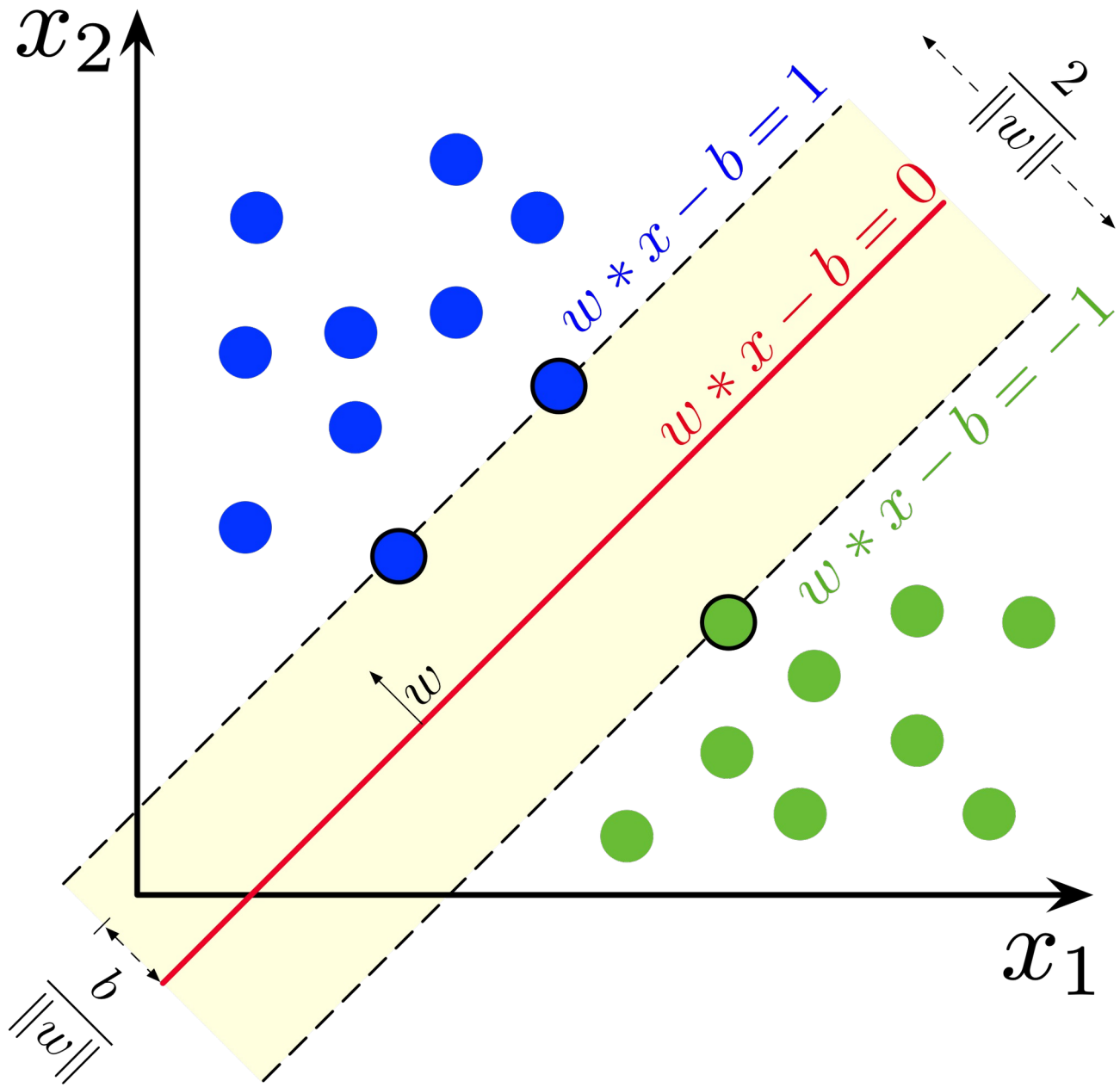
Training algorithm: Discrete optimization

- many versions

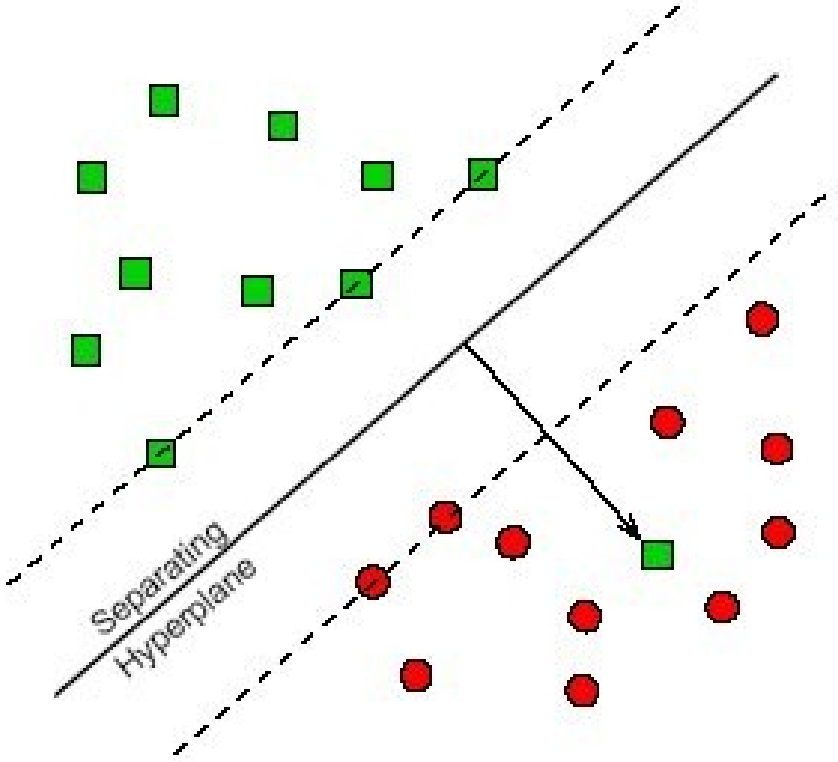


Drawbacks:

- very sensitive to outliers and noise
- does not consider the shape and the size of the training set


x_2
x_1
$w * x - b = 1$
$w * x - b = 0$
$w * x - b = -1$
w
$\frac{2}{\|w\|}$
$\frac{b}{\|w\|}$

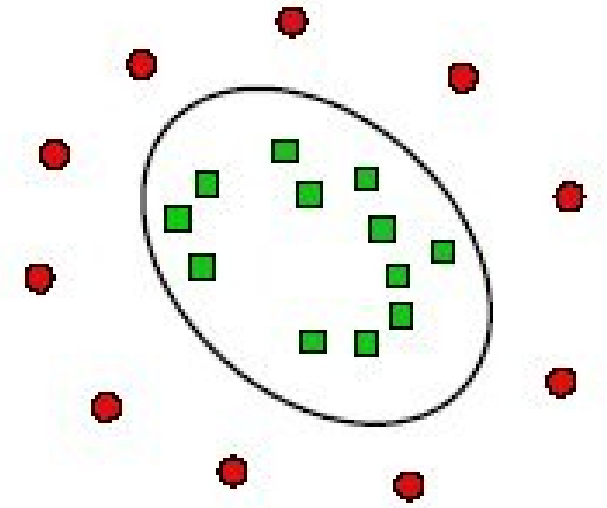
Soft margin SVM



Cost function penalizes the errors

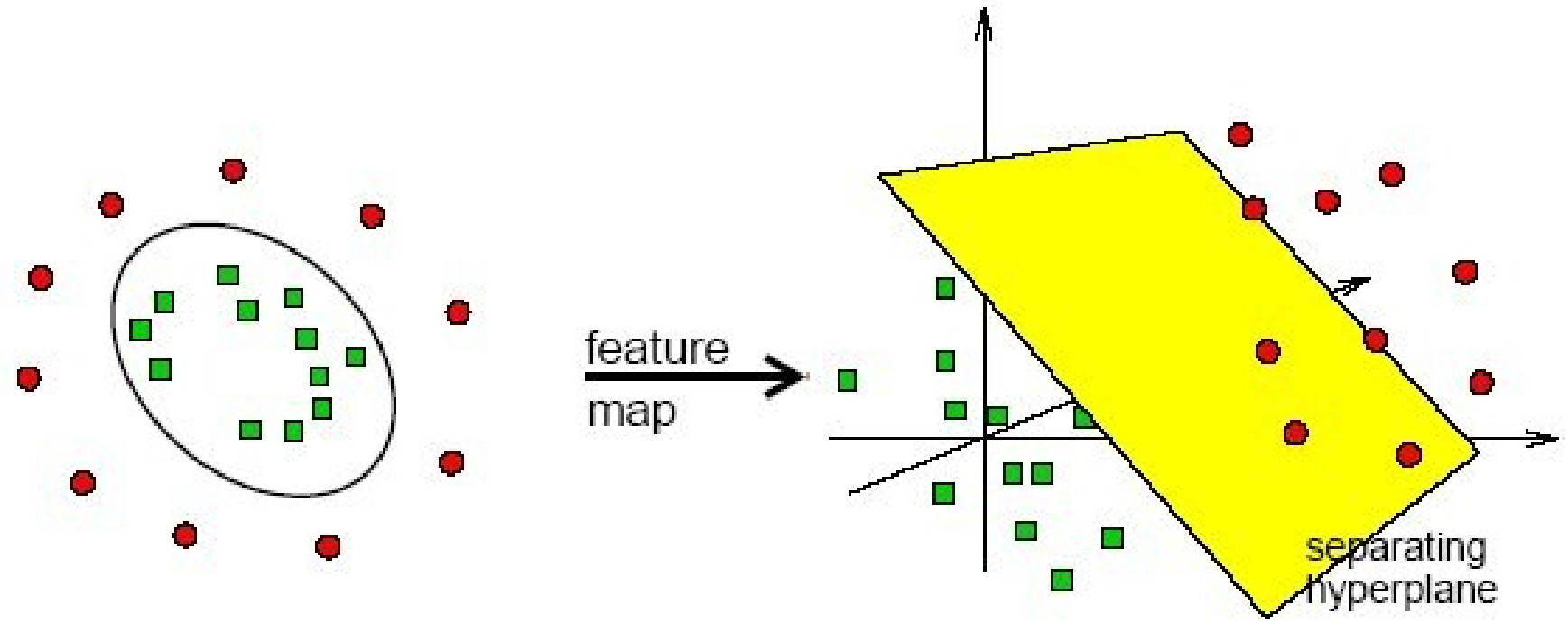
Regularization

SVM for linearly non-separable classes



How to make them linearly separable?

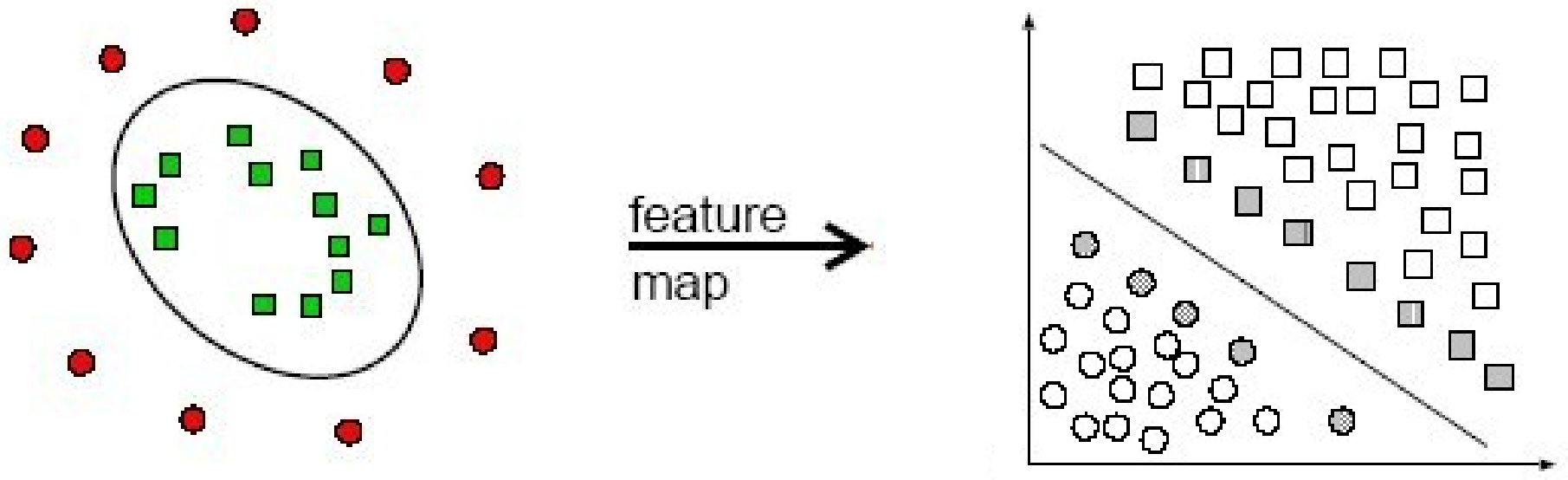
Increasing the number of features



“Curse of dimensionality”

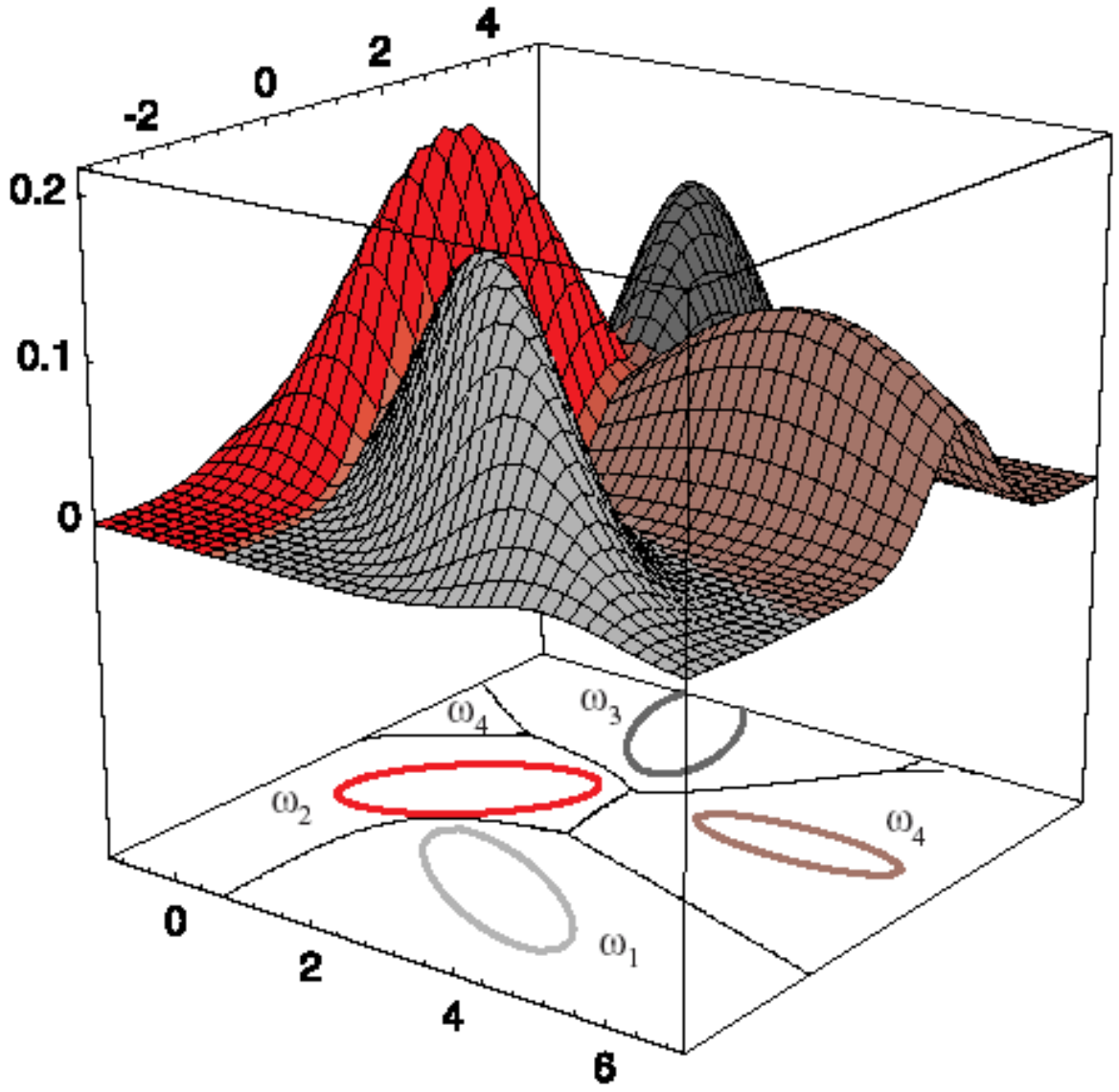
Mapping the features into another space

“The kernel trick”



“Overtraining/undertraining”

Bayesian classifier



Bayesian classifier

Assumption: feature values are random variables

Statistic classifier, the decision is probabilistic

It is based on the **Bayes rule**

The Bayes rule

Class-conditional probability

A posteriori probability

$$P(\omega_j|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_j)P(\omega_j)}{p(\mathbf{x})},$$

A priori probability

Total probability

$$p(\mathbf{x}) = \sum_{j=1}^{c} p(\mathbf{x}|\omega_j)P(\omega_j).$$

Bayesian classifier

Main idea: maximize posterior probability

$$P(\omega_j|\mathbf{x})$$

Since it is hard to do directly, we rather maximize

$$p(\mathbf{x}|\omega_j)P(\omega_j)$$

In case of equal priors, we maximize only

$$p(\mathbf{x}|\omega_j)$$

How to estimate $p(\mathbf{x}|\omega_j)P(\omega_j)$?

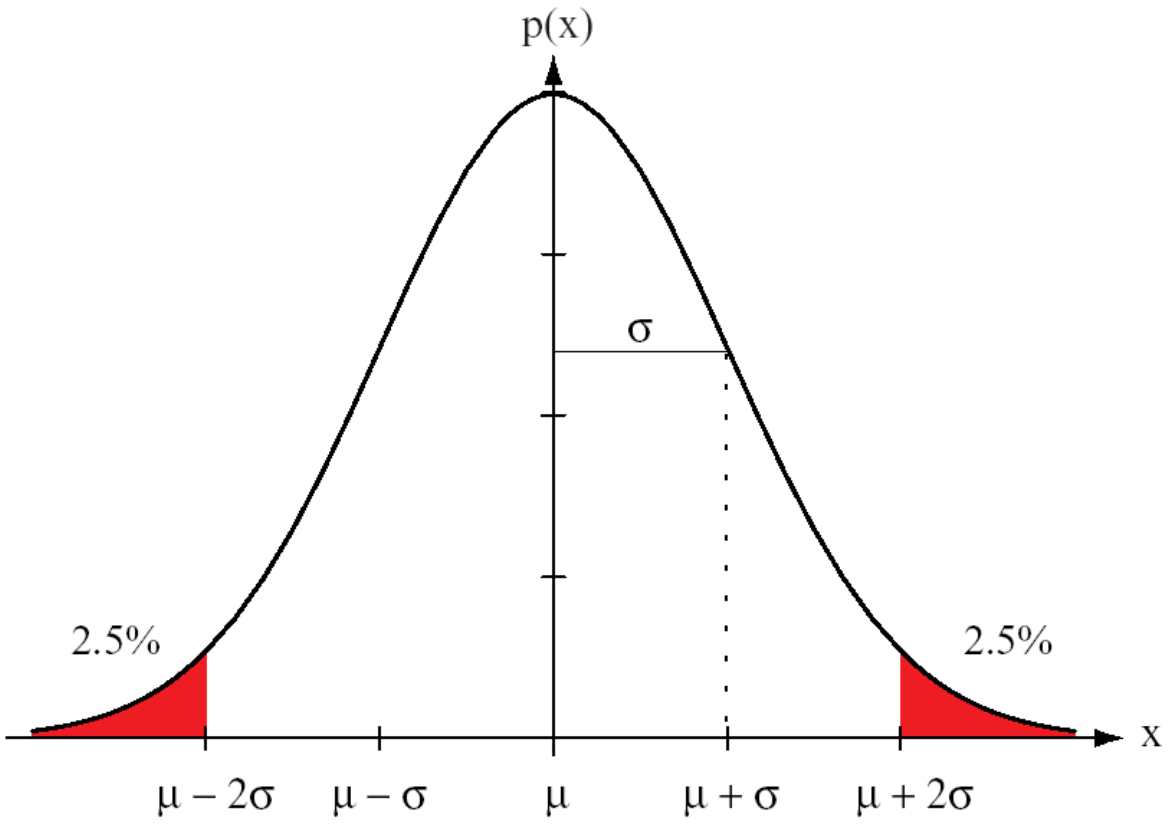
$P(\omega_j)$

- From the case studies performed before (OCR, speech recognition)
- From the occurence in the training set
- Assumption of equal priors

$p(\mathbf{x}|\omega_j)$

- Parametric estimate (assuming pdf is of a known form, e.g. Gaussian)
- Non-parametric estimate (pdf is unknown or too complex)

Parametric estimate of Gaussian $p(\mathbf{x}|\omega_j)$

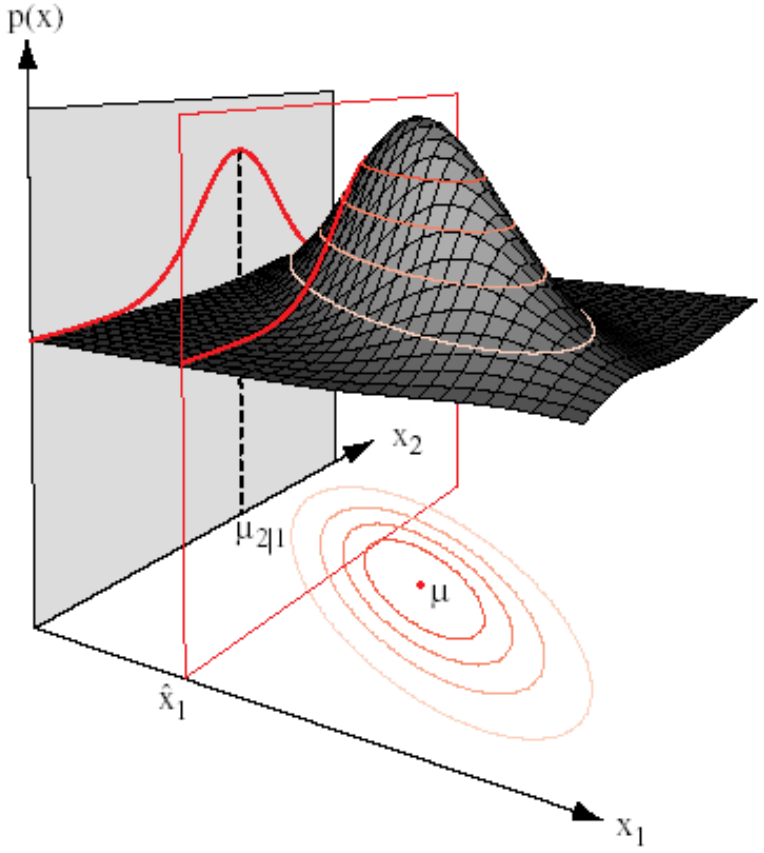


$$\hat{\mu} = \frac{1}{n}\sum_{k=1}^{n} x_k$$

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{k=1}^{n}(x_k - \hat{\mu})^2$$

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right]$$

d-dimensional Gaussian pdf

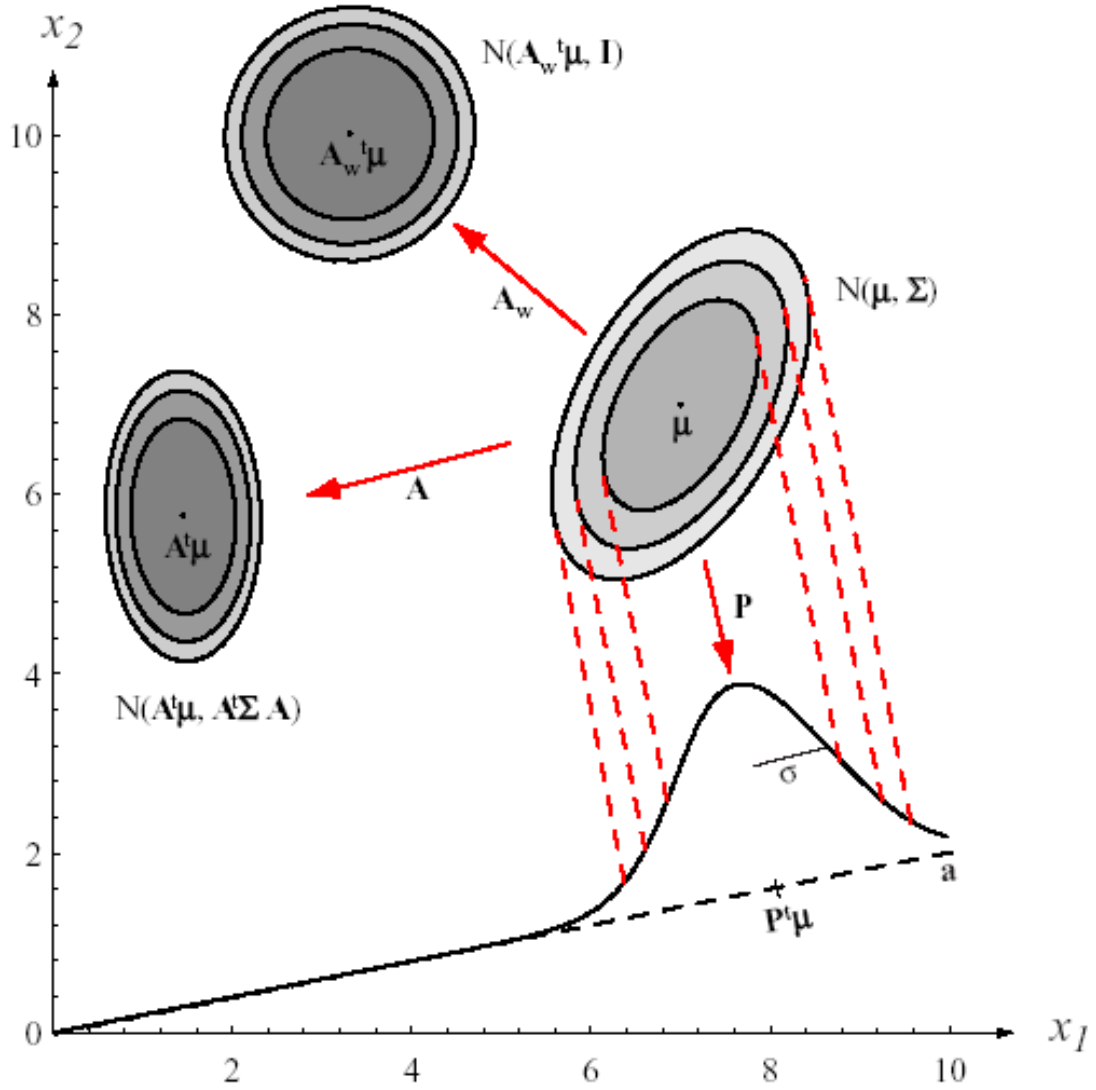


$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{k=1}^{n}\mathbf{x}_k$$

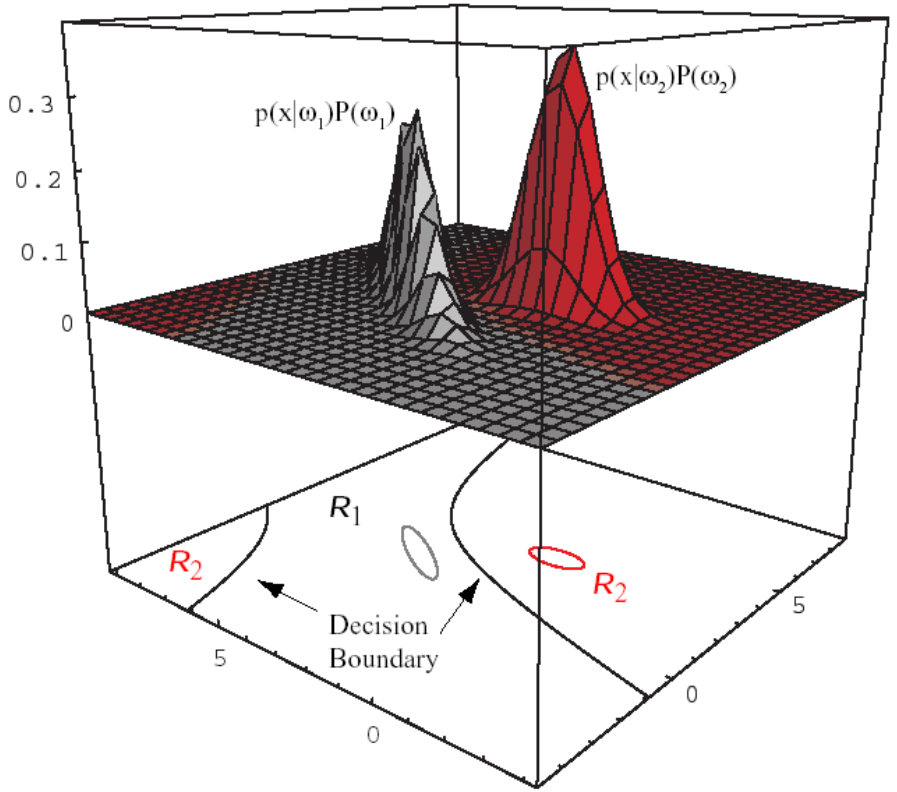
$$\hat{\boldsymbol{\Sigma}} = \frac{1}{n}\sum_{k=1}^{n}(\mathbf{x}_k - \hat{\boldsymbol{\mu}})(\mathbf{x}_k - \hat{\boldsymbol{\mu}})^t.$$

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}}\exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^t\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right]$$

The role of covariance matrix



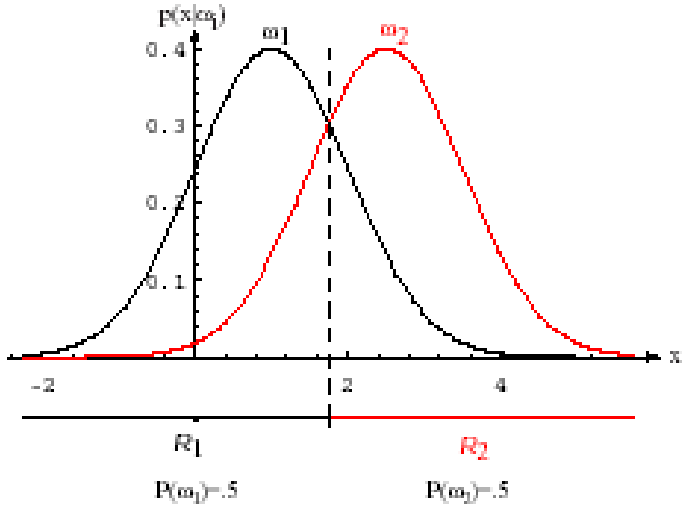
Two-class Gaussian case in 2D

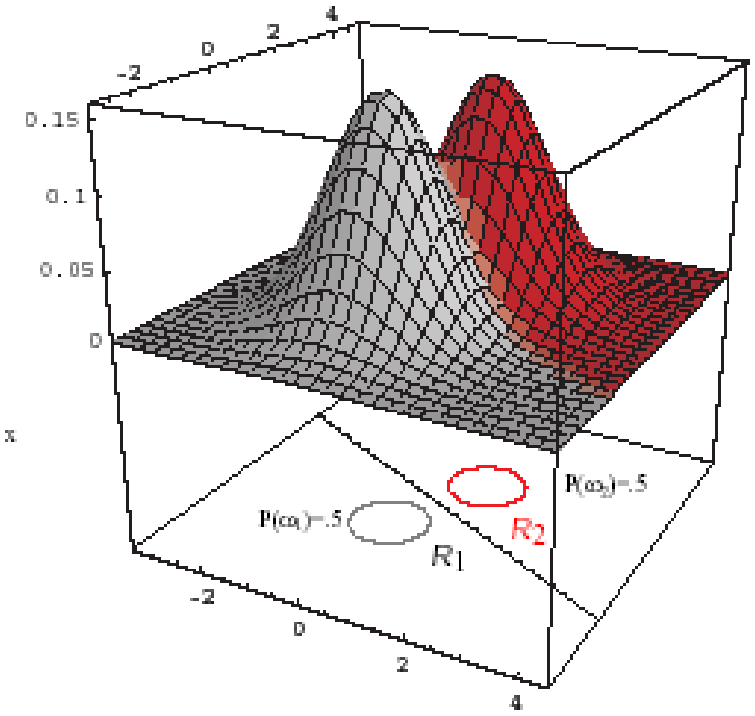


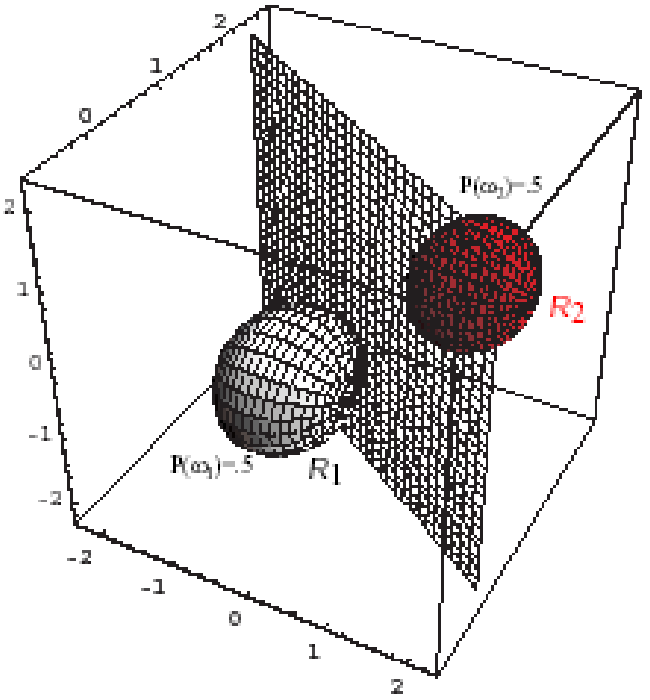
$$g_i(\mathbf{x}) = -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^t\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i) - \frac{d}{2}\ln 2\pi - \frac{1}{2}\ln|\boldsymbol{\Sigma}_i| + \ln P(\omega_i).$$

Classification = comparison of two Gaussians

Two-class Gaussian case – Equal cov. mat.







$$g_i(\mathbf{x}) = -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^t\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_i) + \ln P(\omega_i)$$

Linear decision boundary

Equal priors $P(\omega_j)$

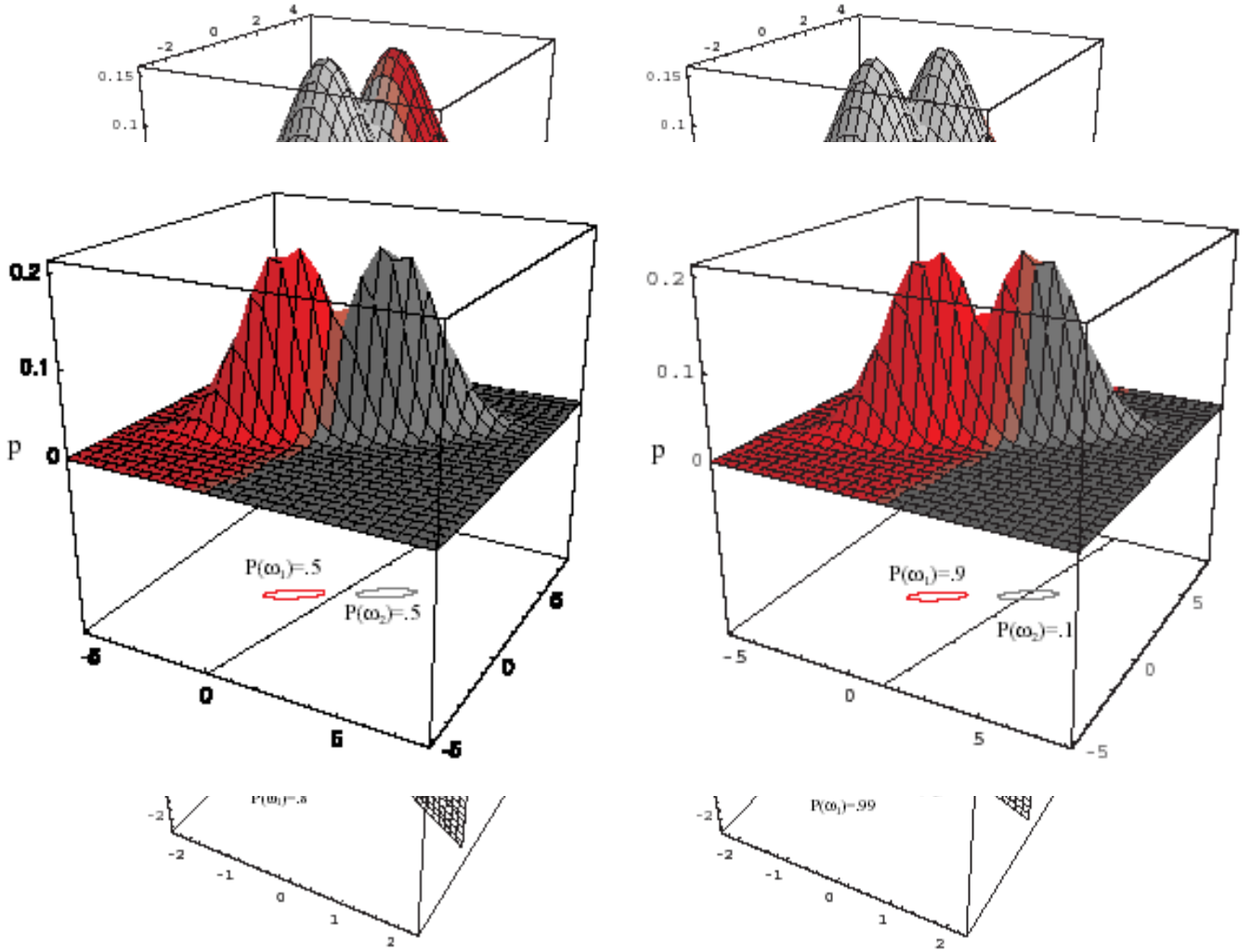
max $g_i(\mathbf{x}) = -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^t\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)$.

min $(\mathbf{x}-\boldsymbol{\mu}_i)^t\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)$.

Classification by minimum **Mahalanobis distance**

If the cov. mat. is diagonal with equal variances then we get **“standard”** minimum distance rule

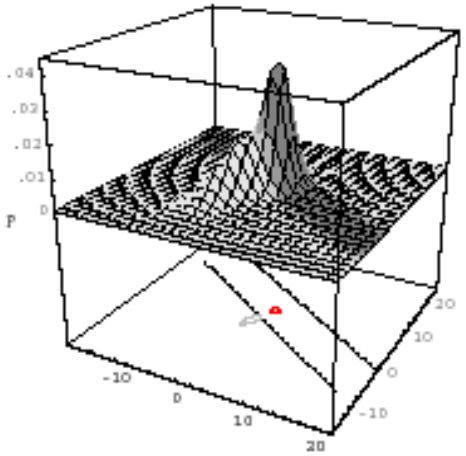
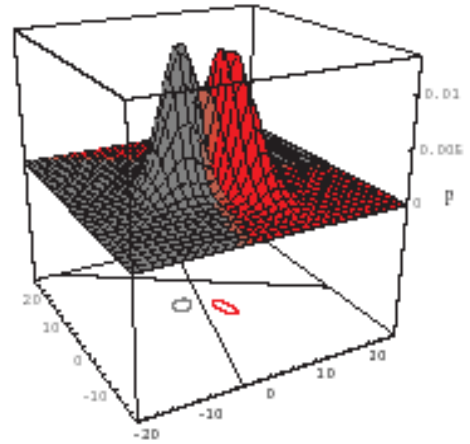
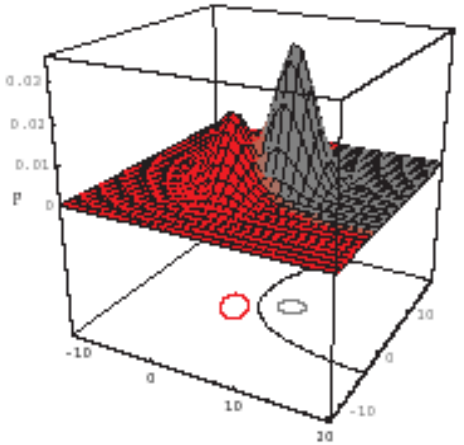
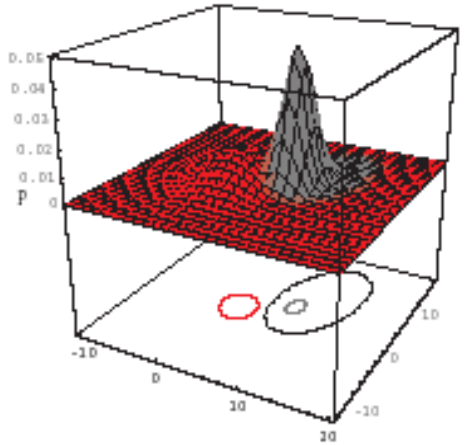
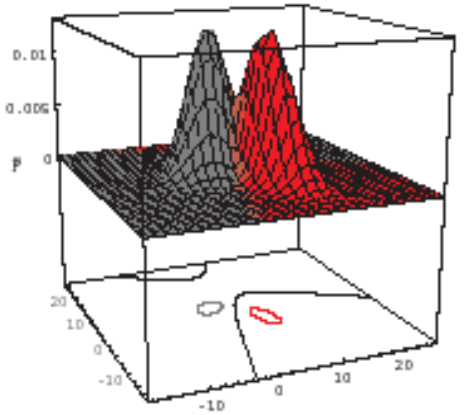
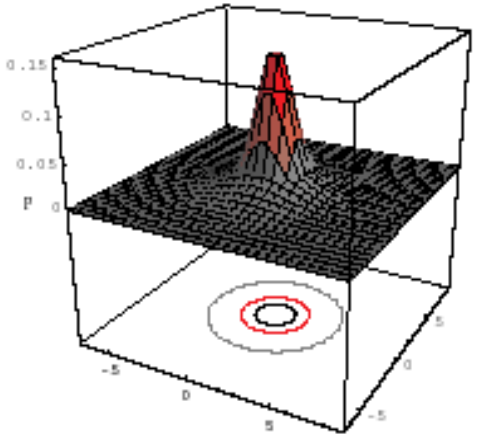
Non-equal priors



Linear decision boundary still preserved

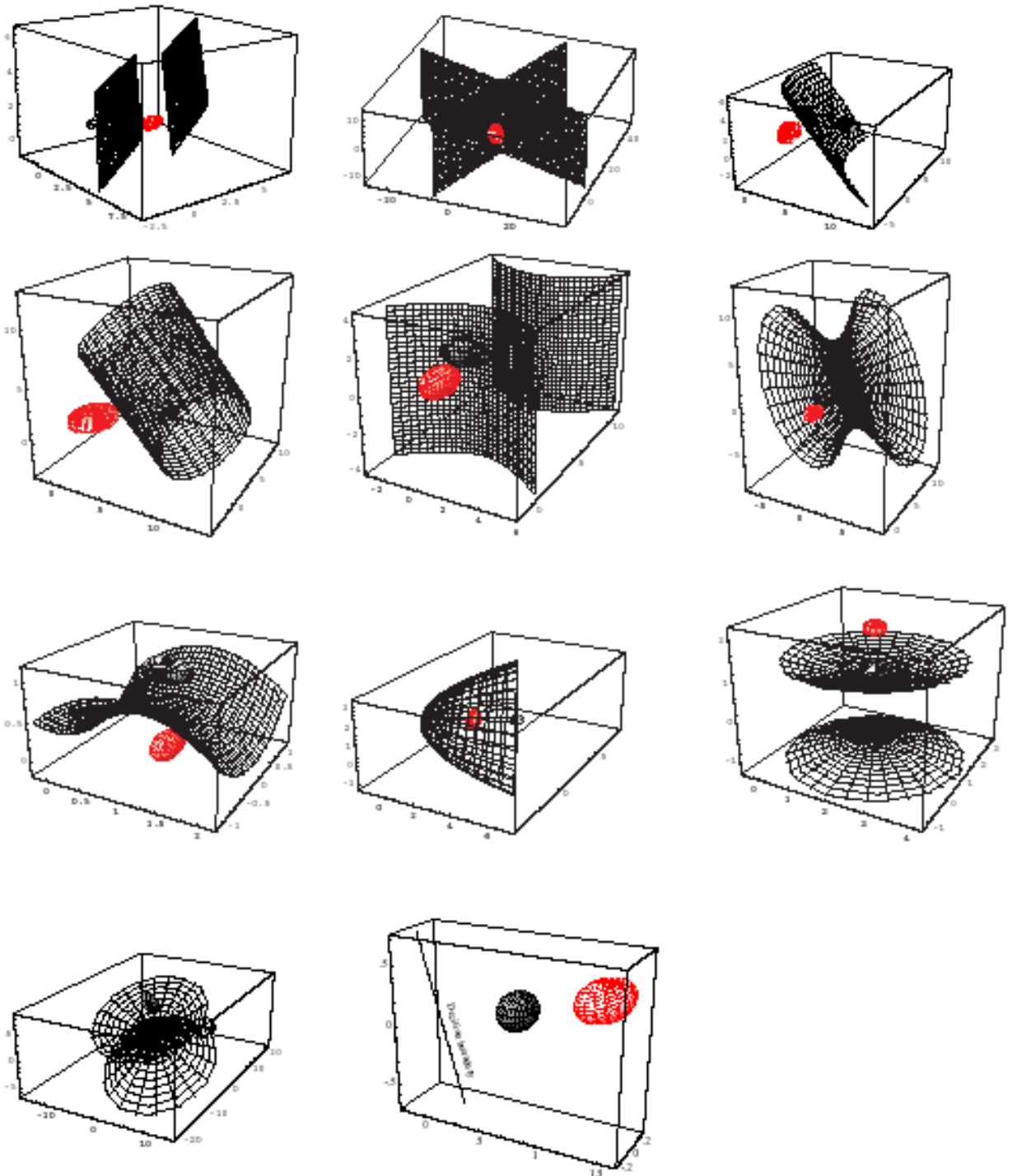
General G case in 2D

Decision boundary is a hyperquadric

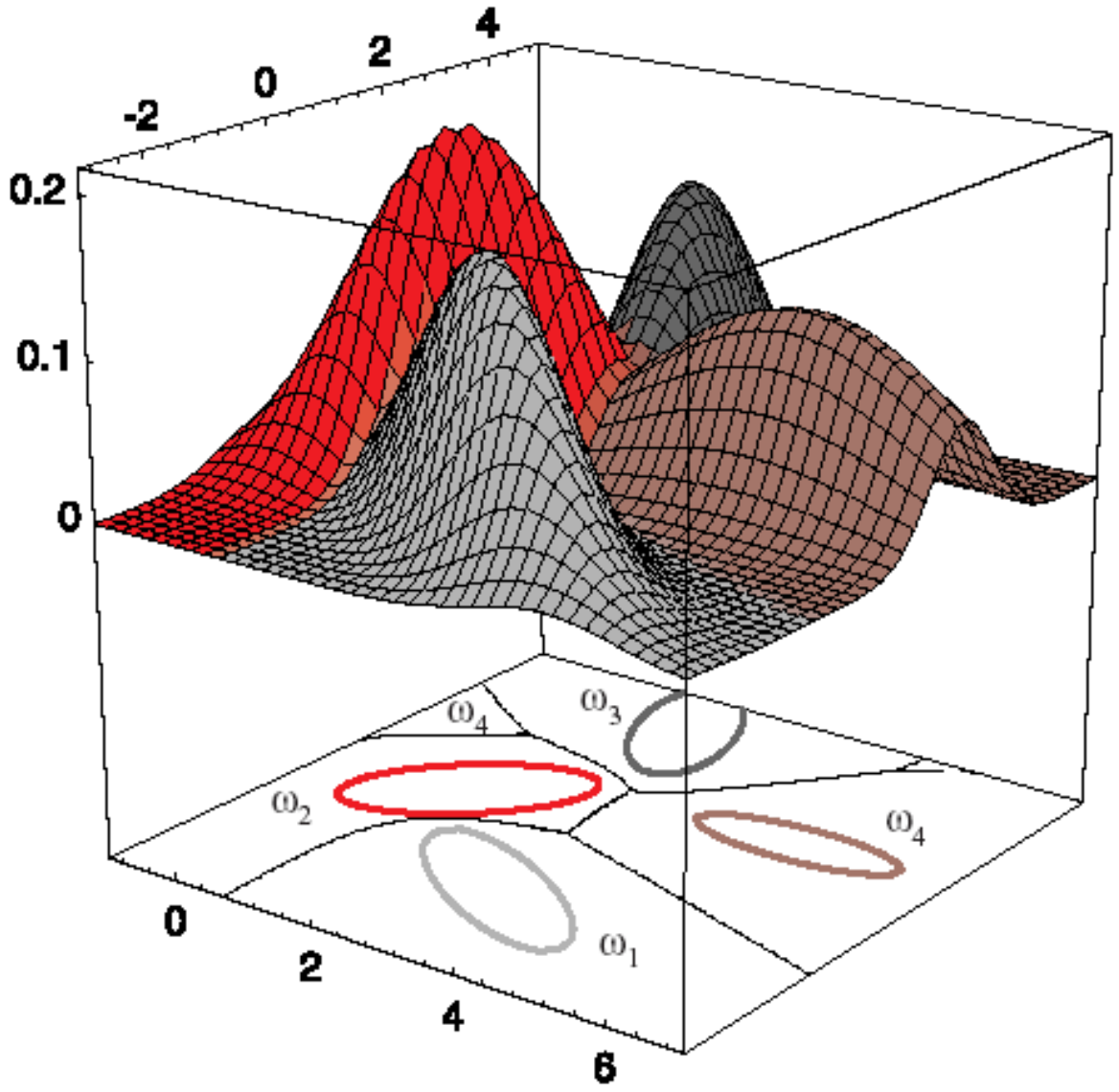







General G case in 3D

Decision boundary is a hyperquadric



More classes, Gaussian case in 2D



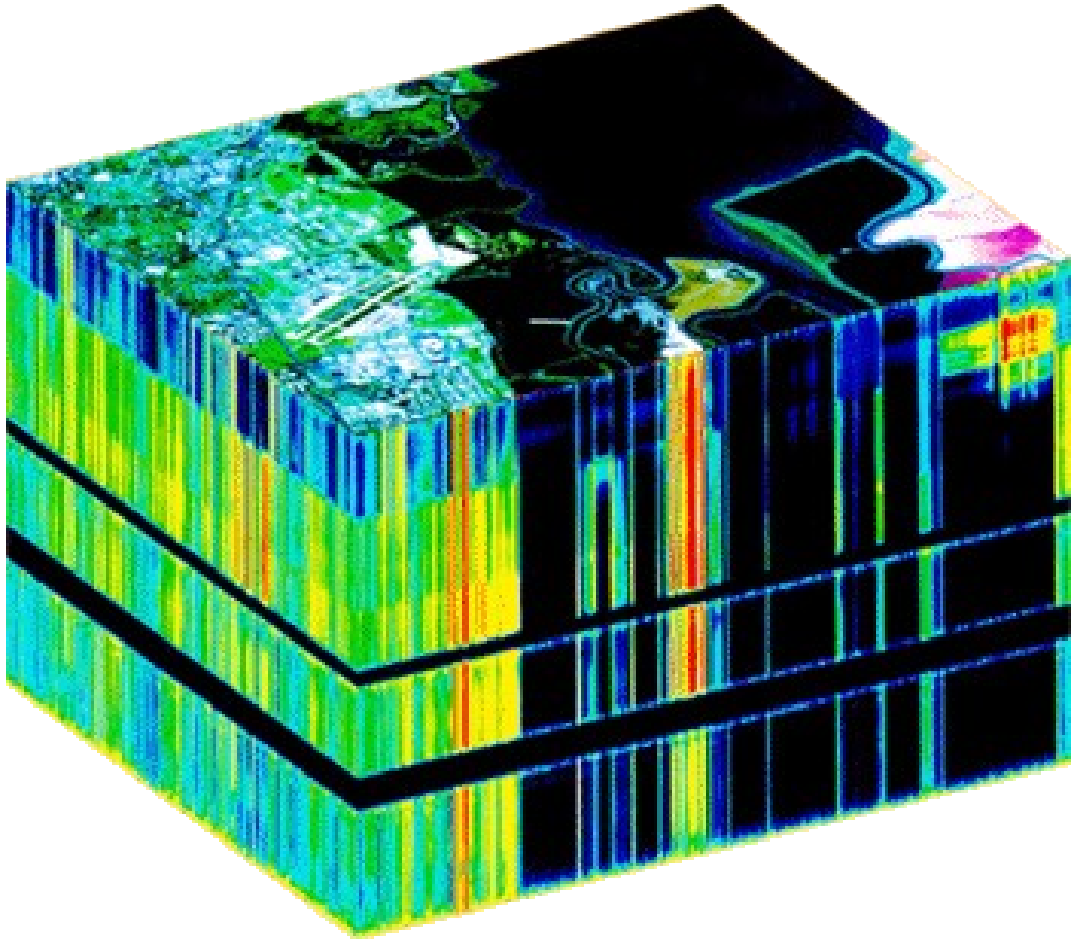
Applications of Bayesian classifier

…in multispectral remote sensing

- Objects = pixels
- Features = pixel values in the spectral bands (from 4 to several hundreds)
- Training set – selected manually by means of thematic maps (GIS), and on-site observation
- Number of classes – typically from 2 to 16

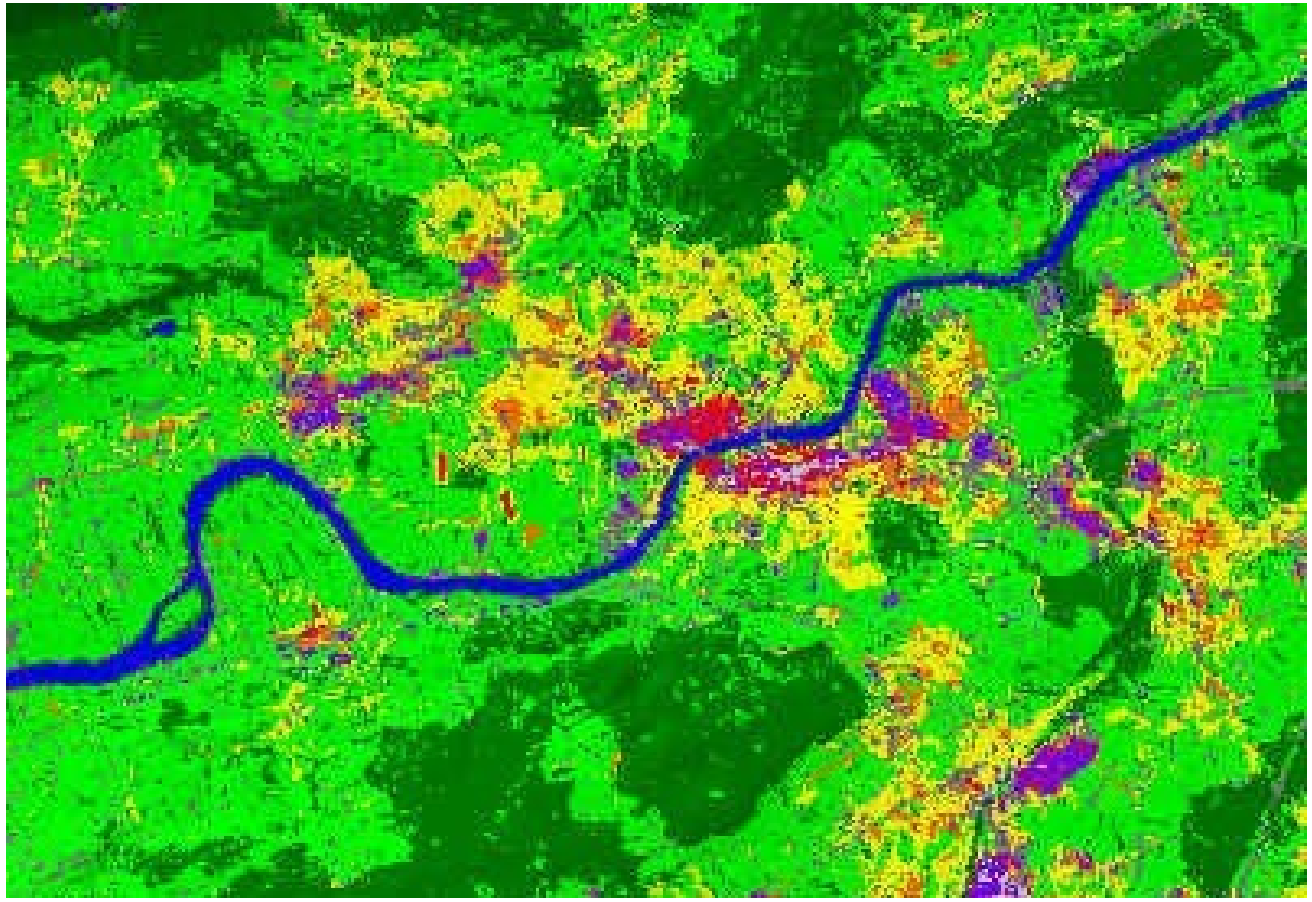
Applications of Bayesian classifier

…in multispectral remote sensing



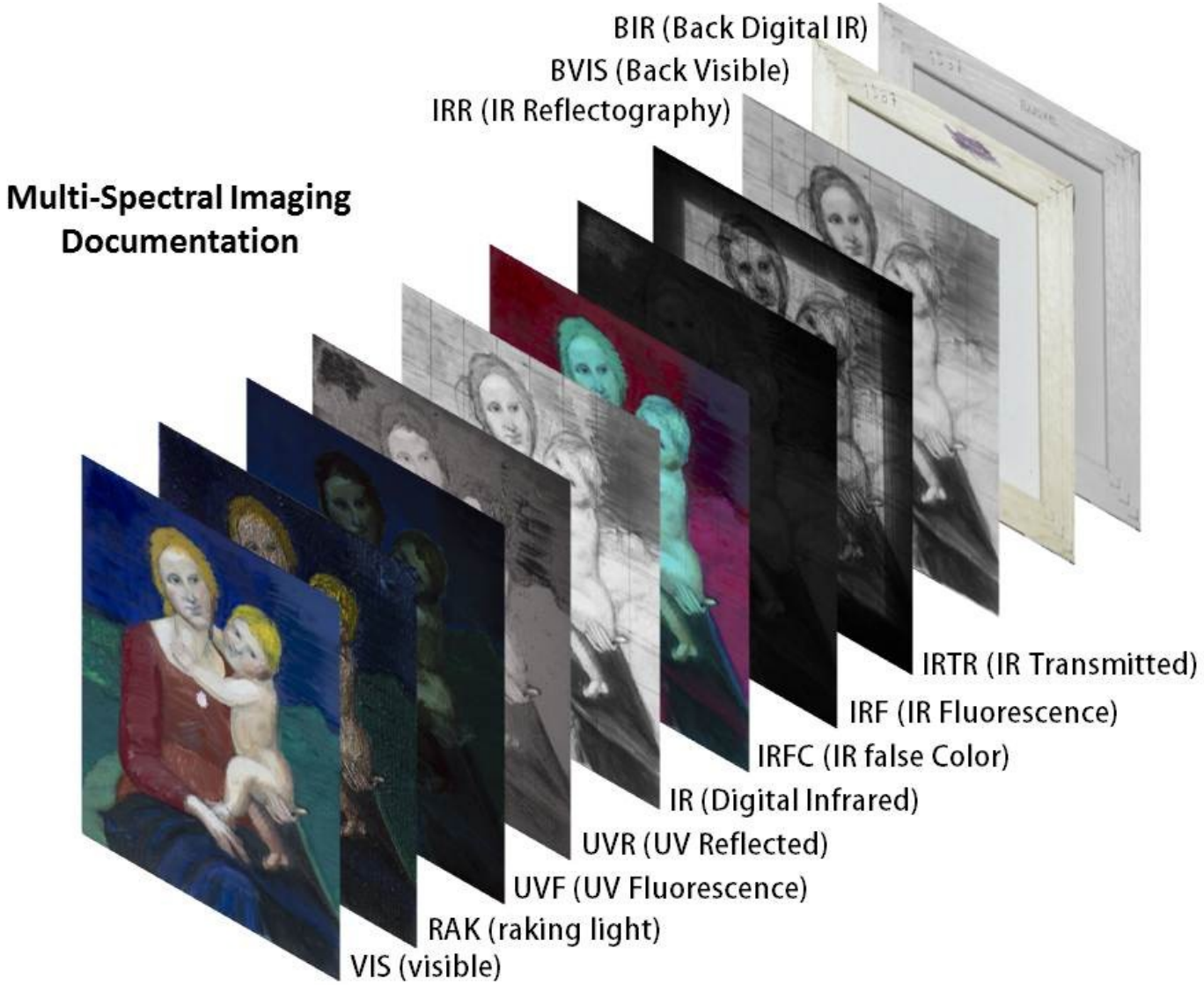
Applications of Bayesian classifier

…in multispectral remote sensing



Applications of Bayesian classifier

…in art



Classification performance

How to evaluate the performance of the classifiers?

- Evaluation on the training set (optimistic error estimate).
- Evaluation on the test set (pessimistic error estimate). Intuitive evaluation is misleading, different errors may have different significance

Confusion matrix

		Predicted		
		Greyhound	Mastiff	Samoyed
Actual	Greyhound	250	25	18
	Mastiff	21	320	24
	Samoyed	22	12	180

Confusion matrix 2 x 2

Machine learing \ Manual counting	True	False
True	True Positive (TP)	False Positive (FP)
False	False Negative (FN)	True Negative (TN)

Equations:

$$\text{False positive rate (FPR)} = \frac{FP}{FP+TN}$$

$$\text{False negative rate (FNR)} = \frac{FN}{FN+TP}$$

$$\text{Sensitivity} = \frac{TP}{TP+FN}$$

$$\text{Specificity} = \frac{TN}{TN+FP}$$

$$\text{Youden index} = \text{Sensitivity} + \text{Specificity} - 1$$

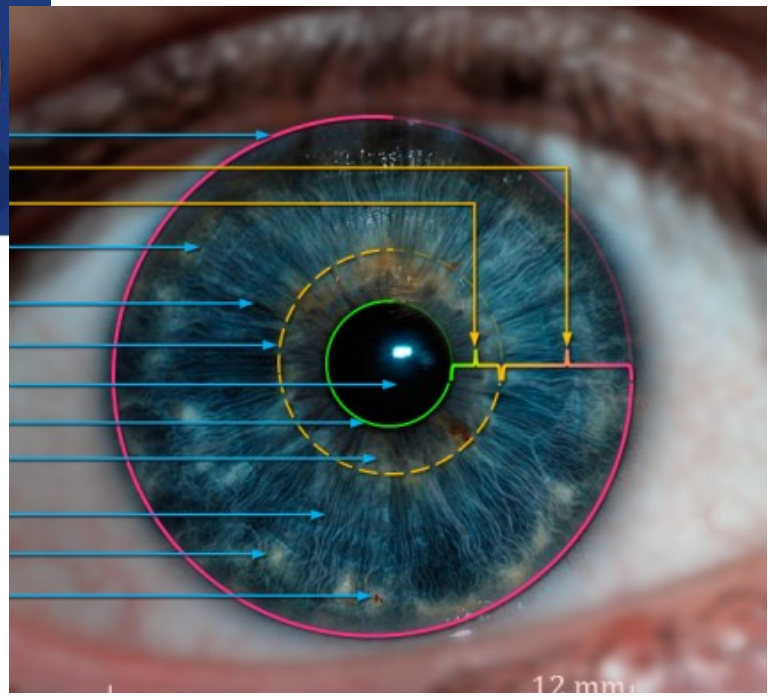
$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Classification performance

How to increase the performance?

- other features
- more features (dangerous – curse of dimensionality!)
- other (larger, better) training sets
- other parametric models
- other classifiers
- **combining different classifiers**

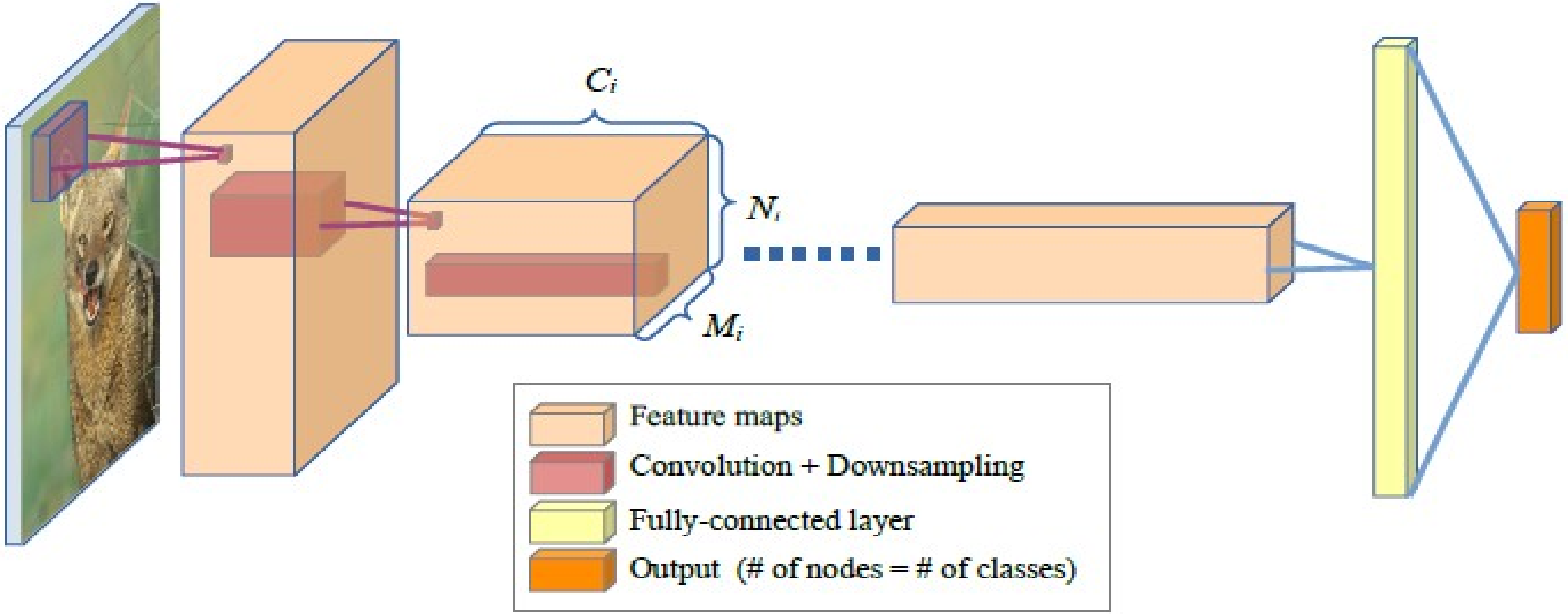
Combining classifiers in biometrics



Limitations of invariant theory

Generic classes, no model of intra-class variability

Convolution networks



Teaching by showing many examples without saying what the features are

Handcrafted vs. Learned features

- Insight
- Explainability
- Training set size
- Debugging
- Concurrent or collaborative approaches?

Unsupervised Classification

Cluster analysis

Training set is not available, No. of classes may not be *a priori* known

What are clusters?

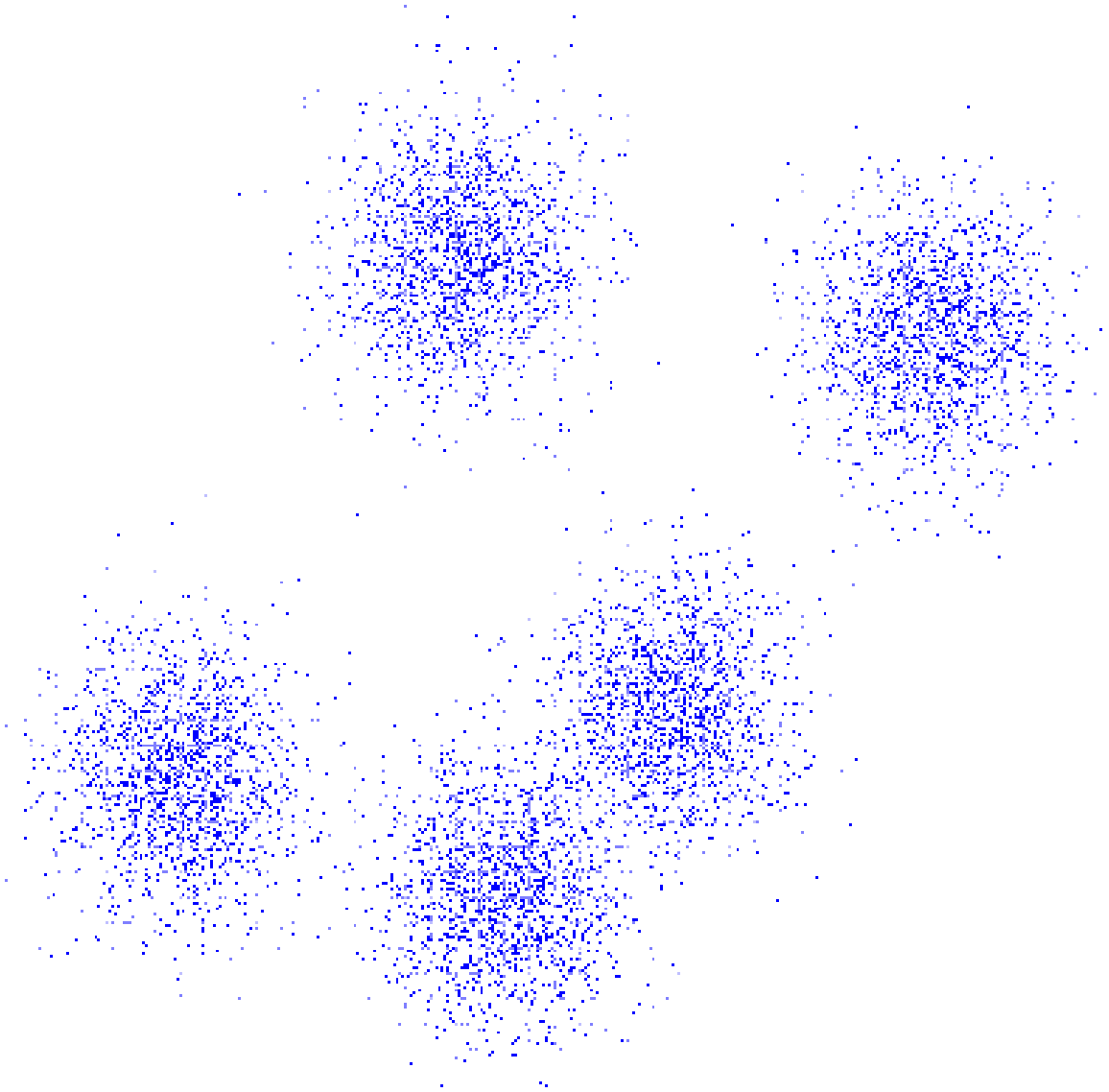
Intuitive meaning

- compact, well-separated subsets

Formal definition

- many attempts, mostly unsatisfactory
  (*t*-connectivity, diameter constraint, ...)
- any partition of the data into disjoint subsets

What are clusters?



How to compare different clusterings?

Ward criterion

Variance measure *J* should be minimized

$$J = \sum_{i=1}^{N} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|^2$$

Drawback – only clusterings with the same *N* can be compared.
Global minimum

J = 0 is reached in the degenerated case.

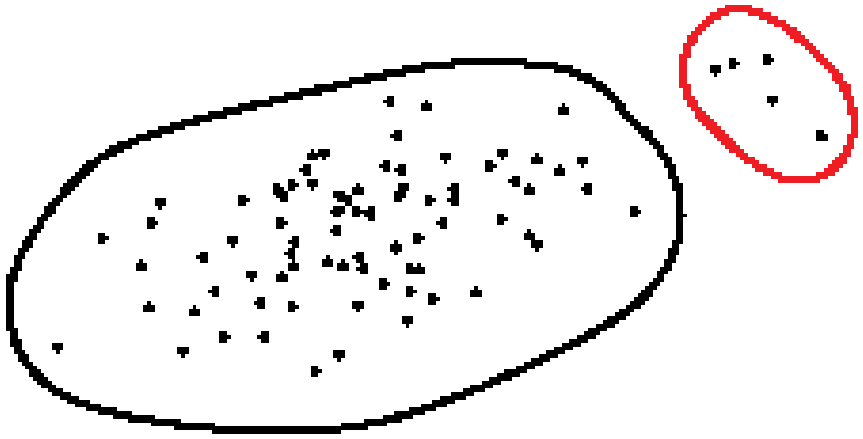
Minimization of *J*

Drawbacks

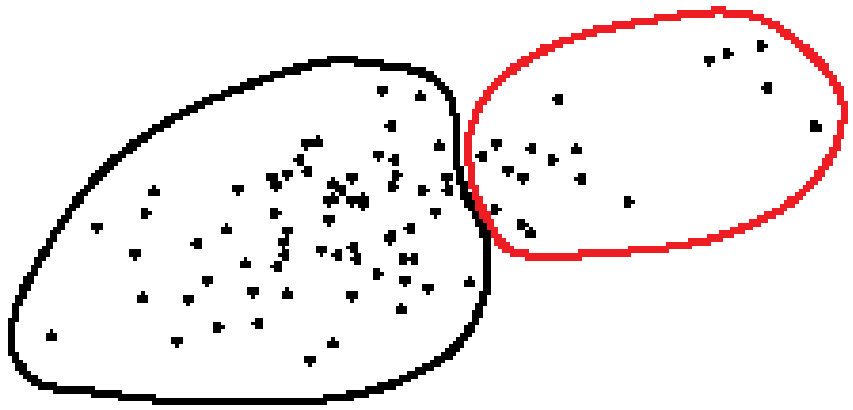
The results are sometimes “intuitively wrong”

because *J* prefers clusters with approx the same size

An example of a „wrong“ result

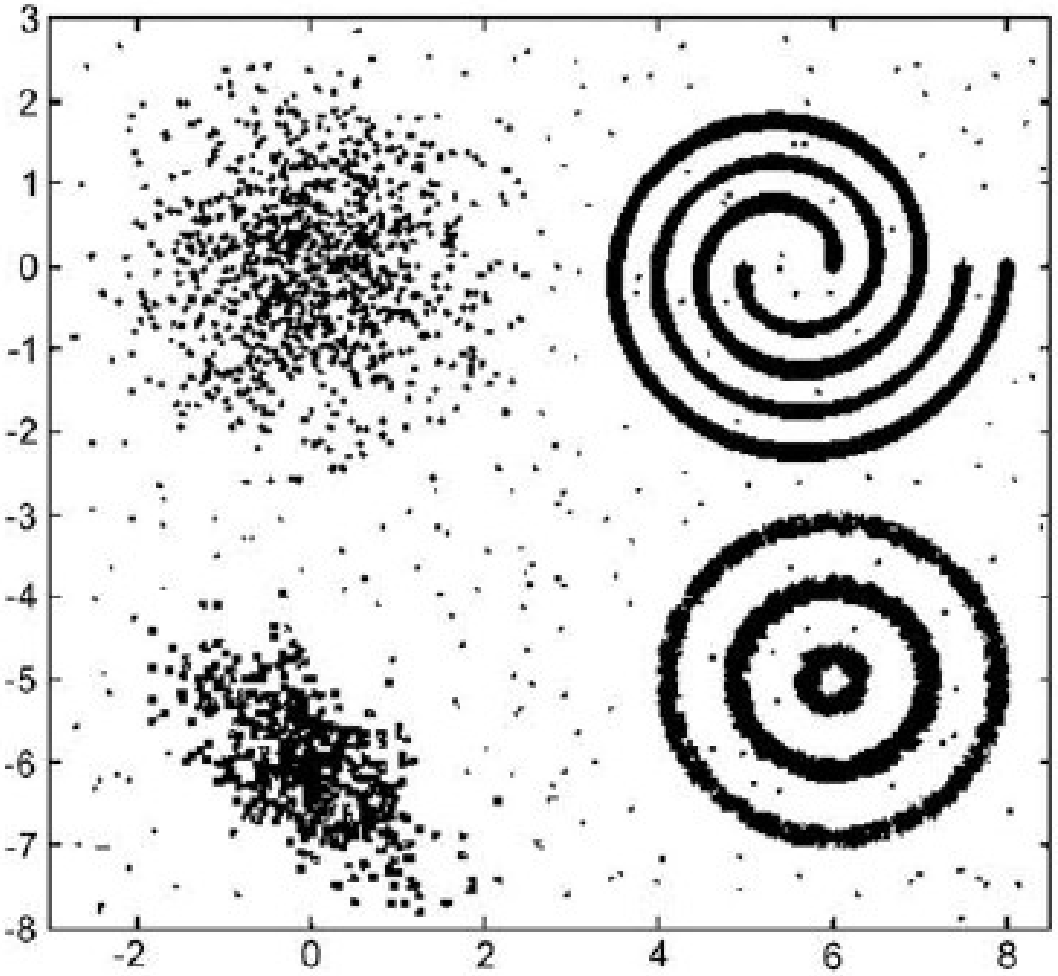


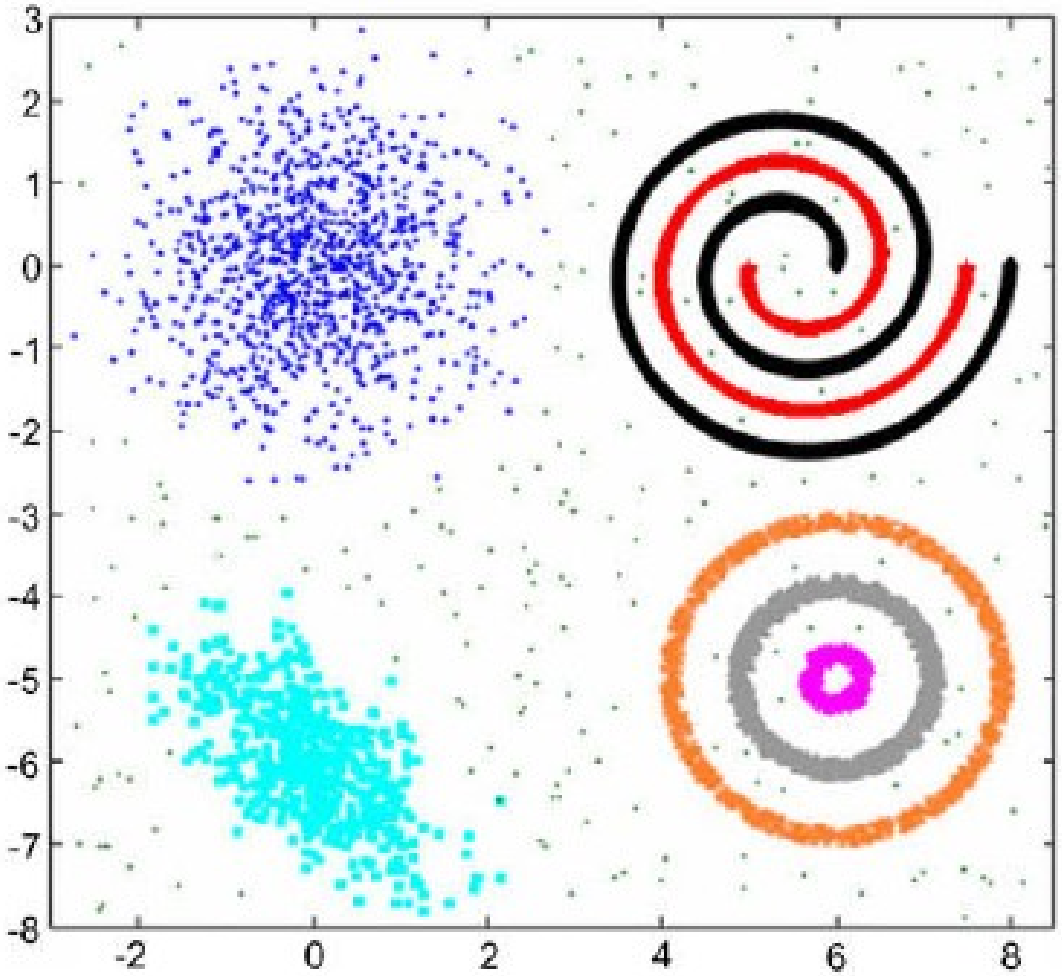
$J_e = large$



$J_e = small$

Drawback – assumes uncorrelated features and “convex” clusters





Clustering techniques

Iterative methods

- typically if *N* is given

Hierarchical methods

- typically if *N* is unknown

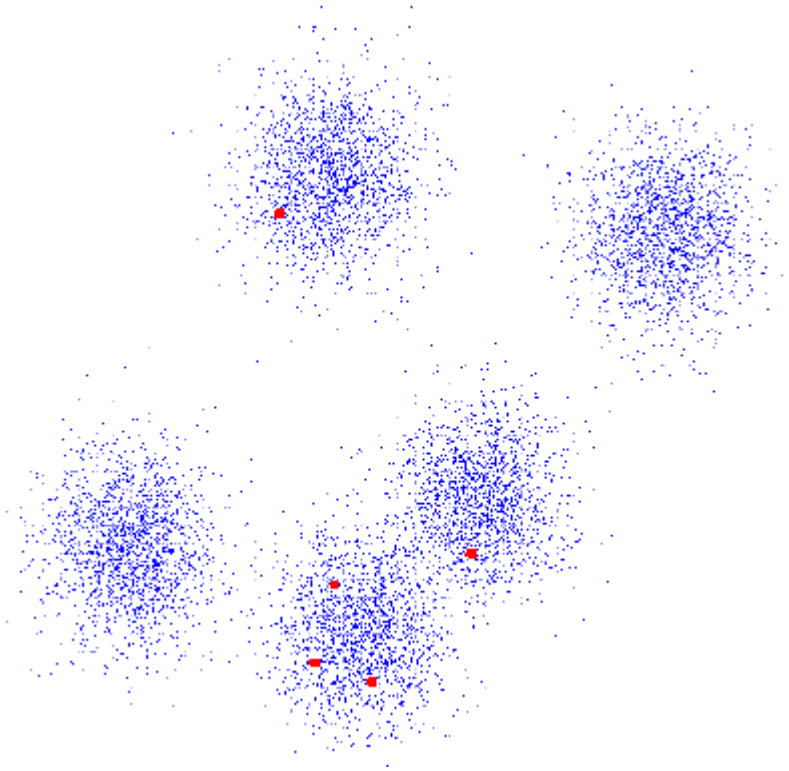
Other methods

- sequential, graph-based, branch & bound, fuzzy, genetic,

  model-based, etc.

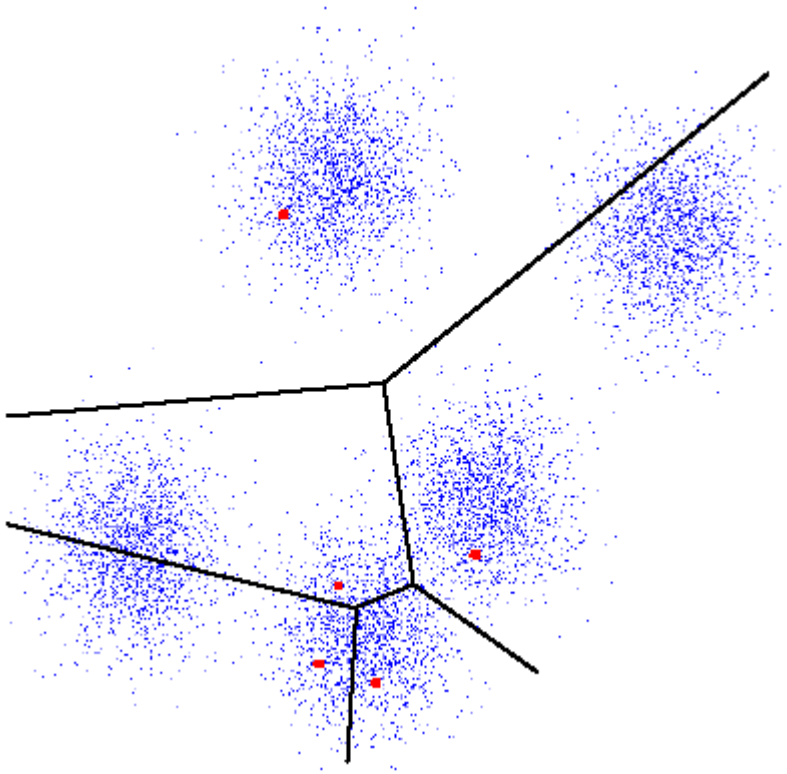
N-means clustering

1. Select *N* initial cluster centroids.



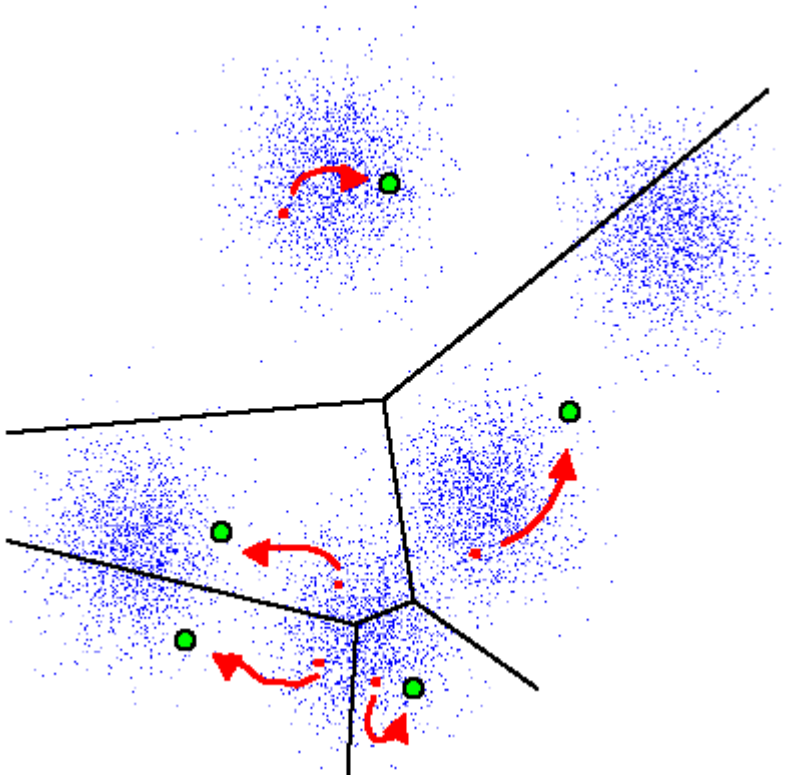
N-means clustering

2. Classify every point *x* according to minimum distance.



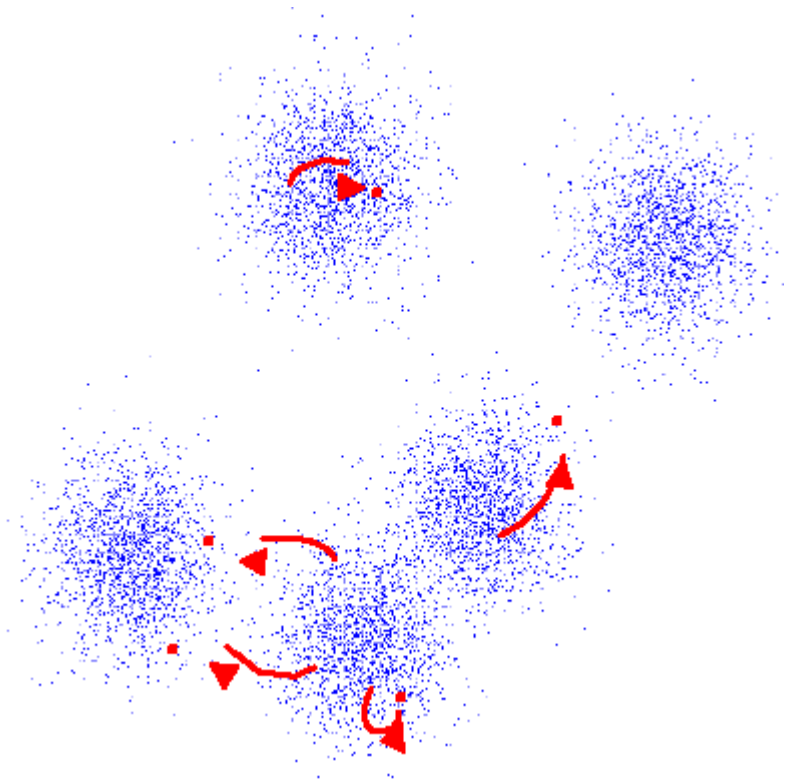
N-means clustering

3. Recalculate the cluster centroids.



N-means clustering

4. If the centroids did not change then STOP else GOTO 2.



N-means clustering

Drawbacks

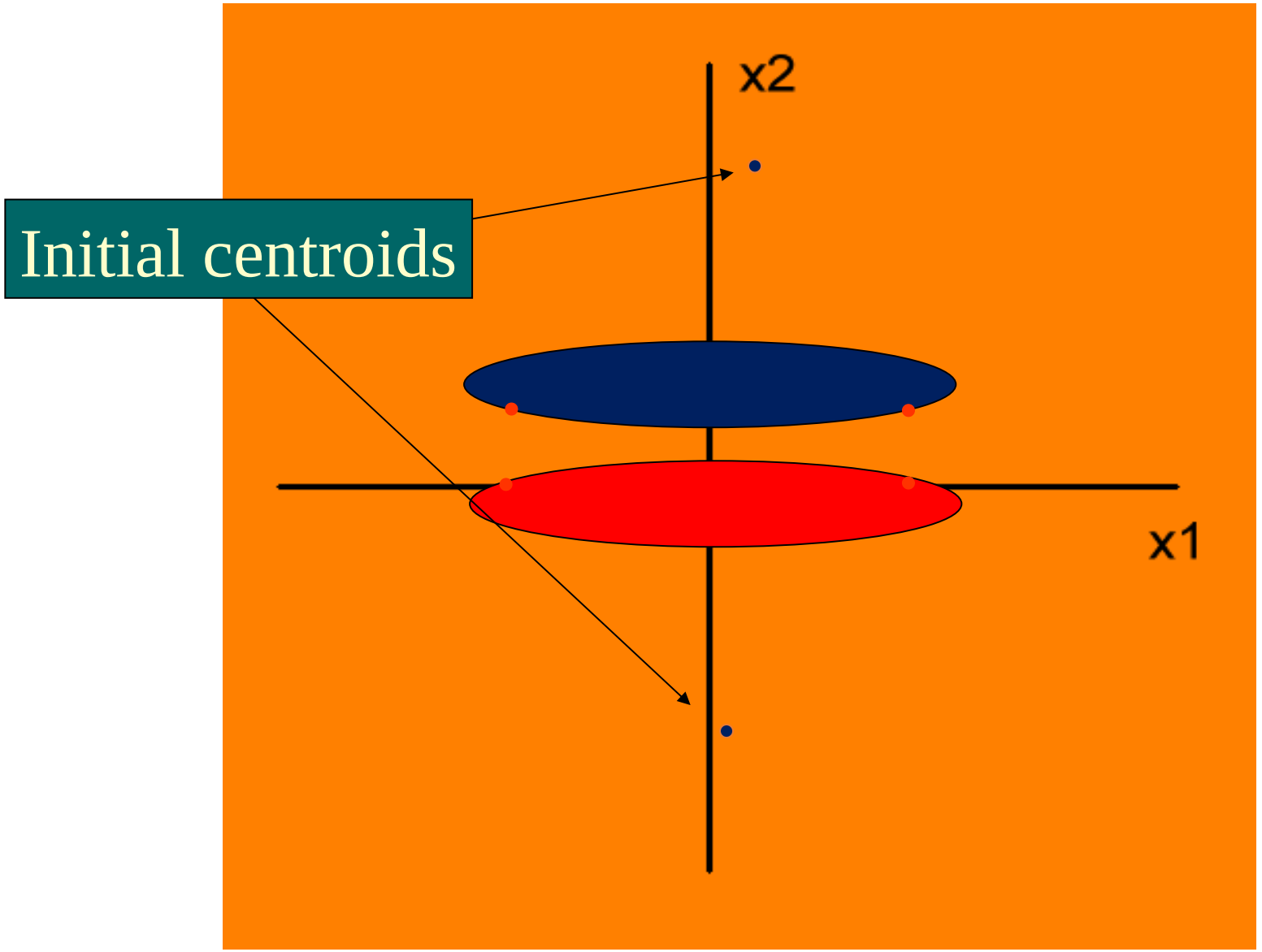
- The result depends on the initialization
- J is not minimized
- The results are sometimes “intuitively wrong”

N-means clustering – an example

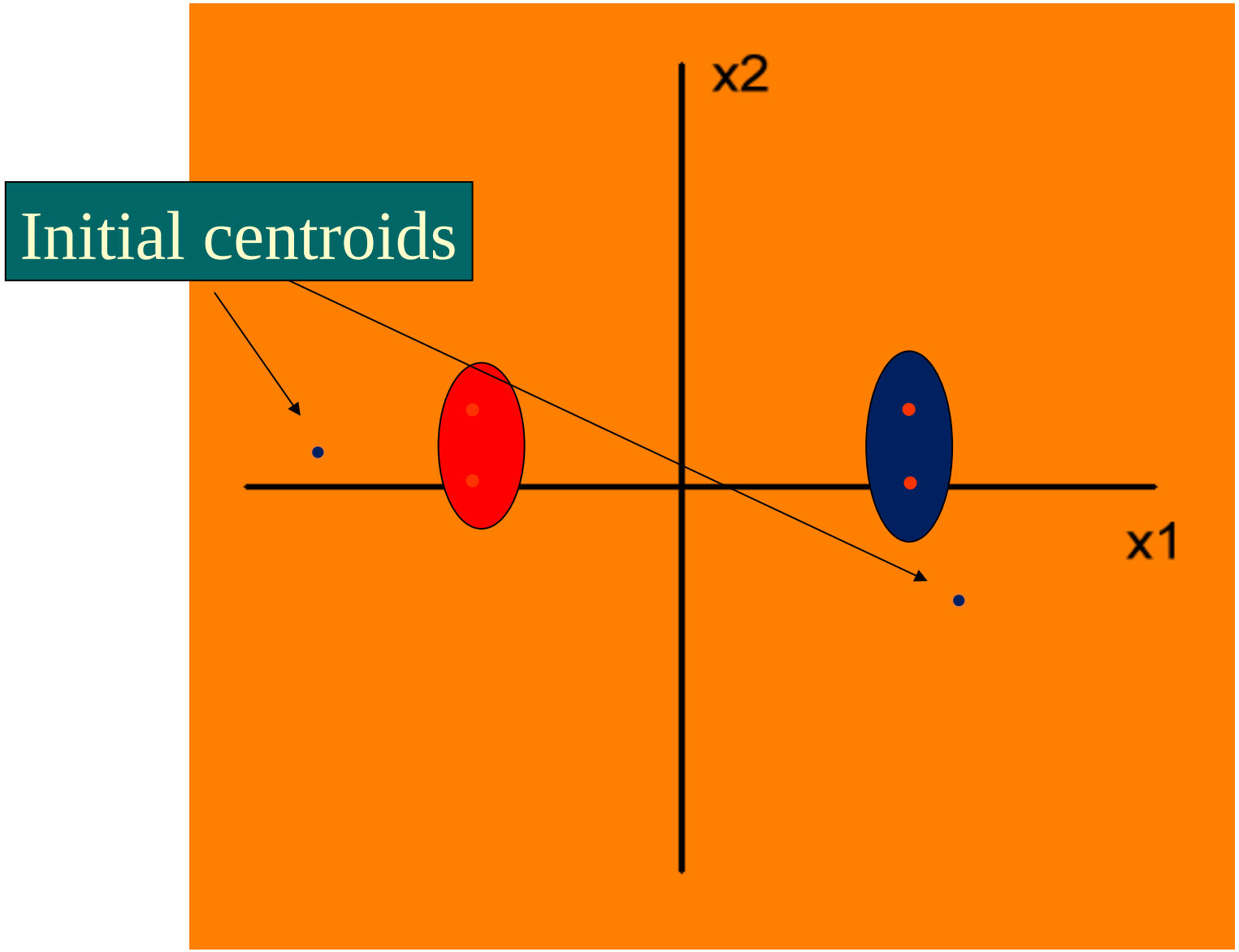
Two features, four points, two clusters ($N = 2$)

Different initializations ✉ different clusterings

N-means clustering – an example

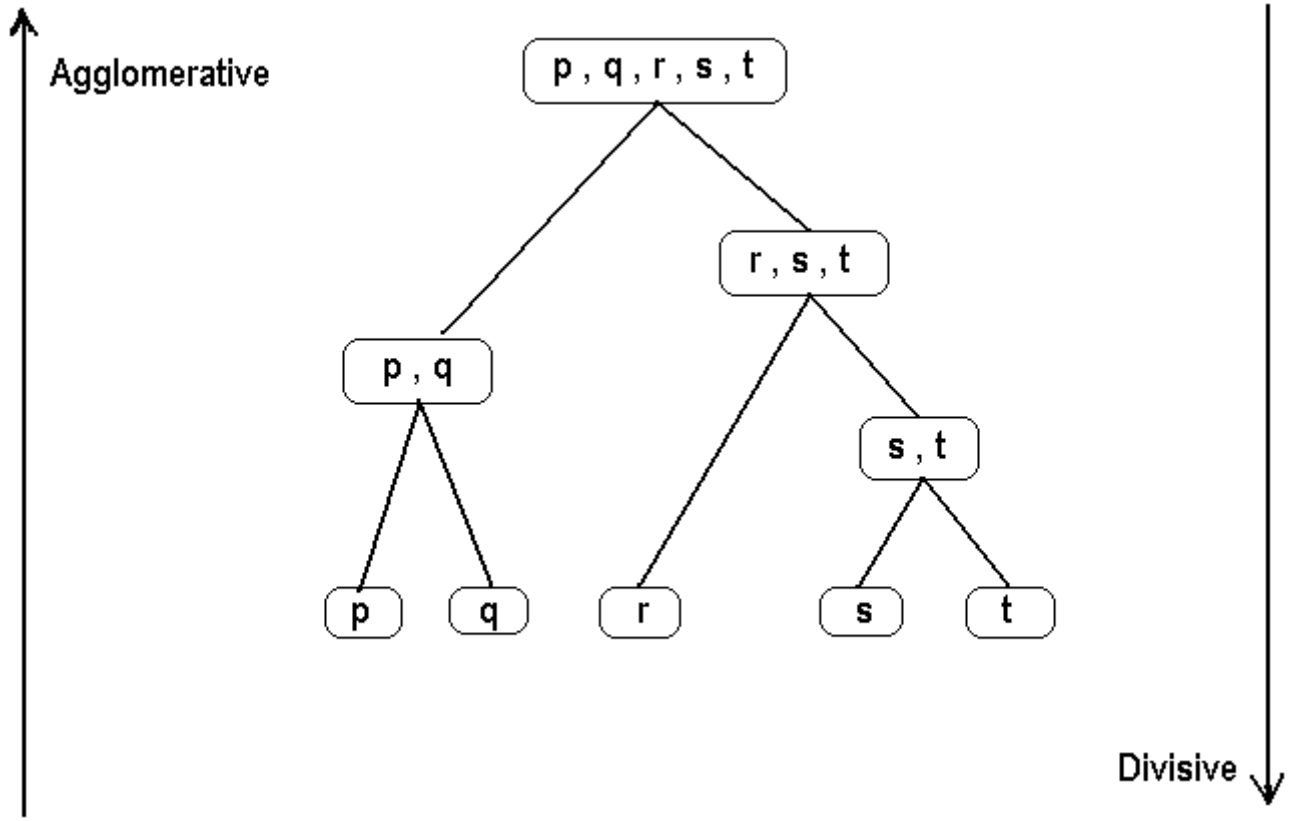


N-means clustering – an example



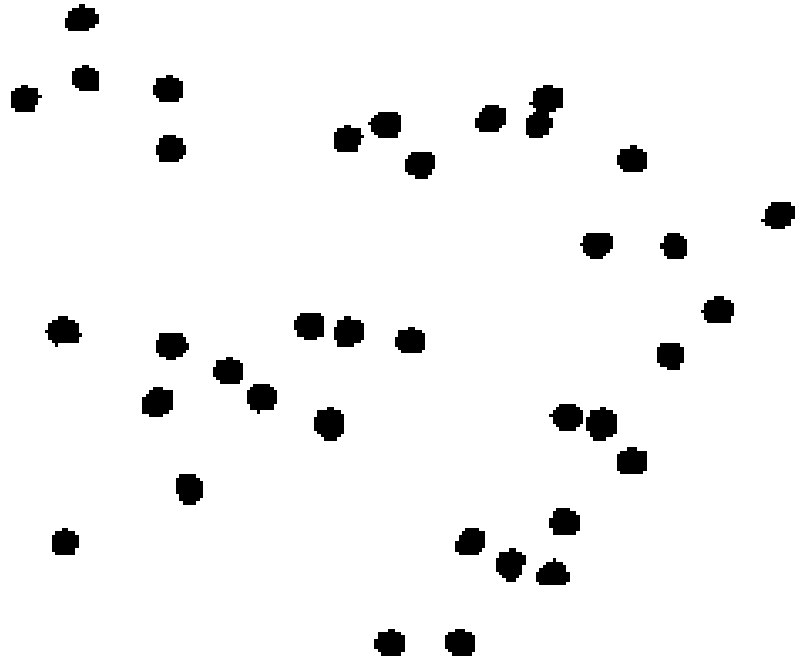
Hierarchical clustering

- Agglomerative clustering
- Divisive clustering



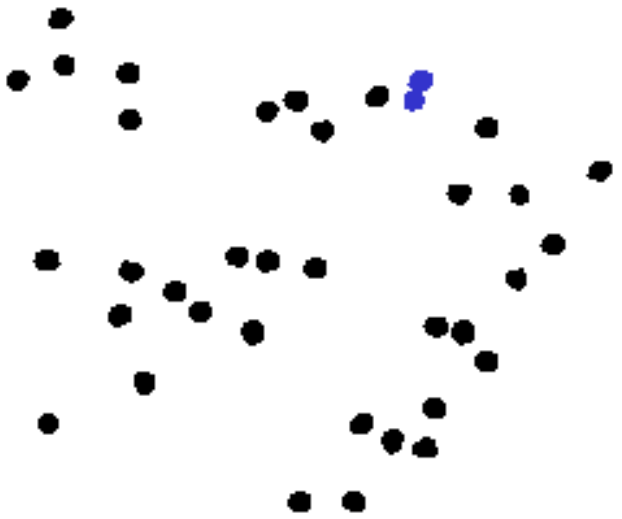
Basic agglomerative clustering

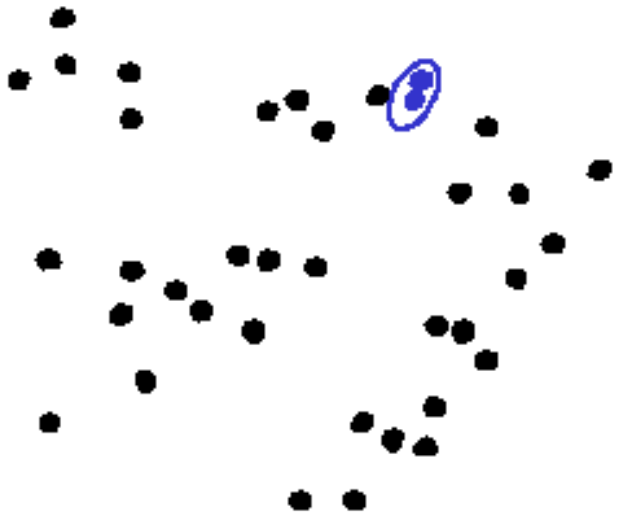
1. Each point = one cluster



Basic agglomerative clustering

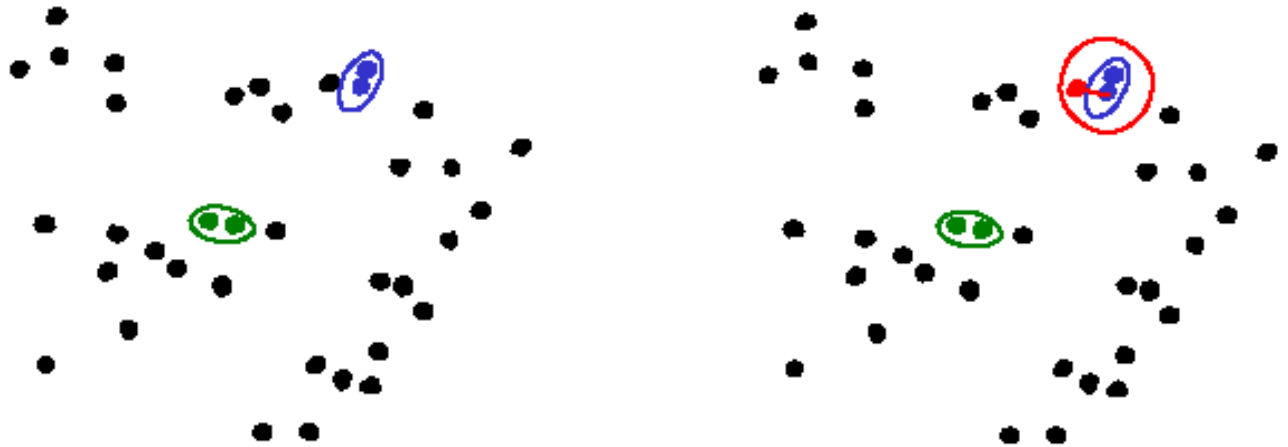
1. Each point = one cluster
2. Find two “nearest” or “most similar” clusters and merge them together





Basic agglomerative clustering

1. Each point = one cluster
2. Find two “nearest” or “most similar” clusters and merge them together
3. Repeat 2 until the stop constraint is reached



Basic agglomerative clustering

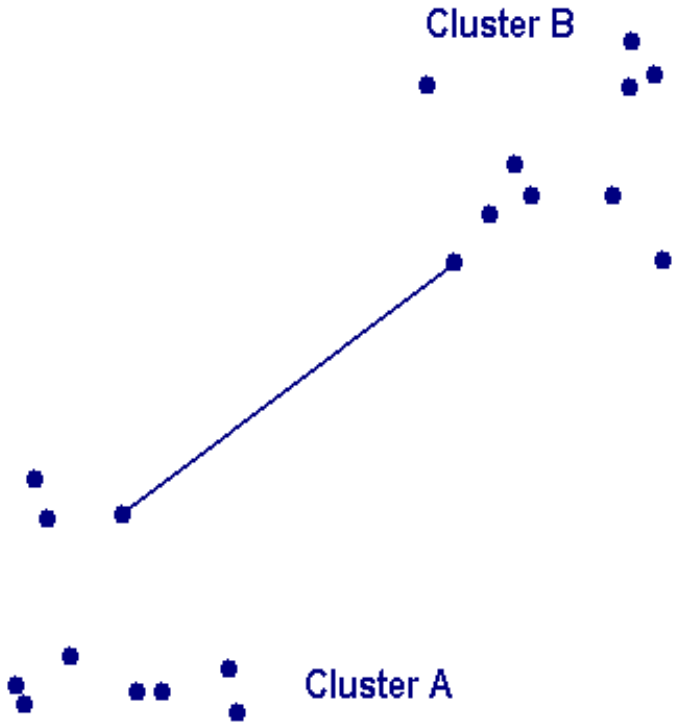
Particular implementations of this method differ from each other by:

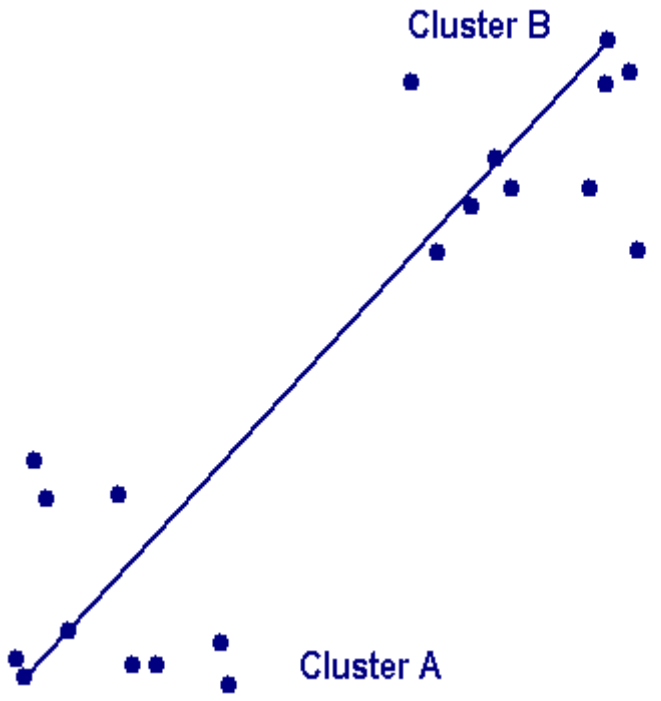
- The STOP constraints
- The distance/similarity measures used

Simple between-cluster distance measures

$$d(A,B) = d(m_1,m_2)$$

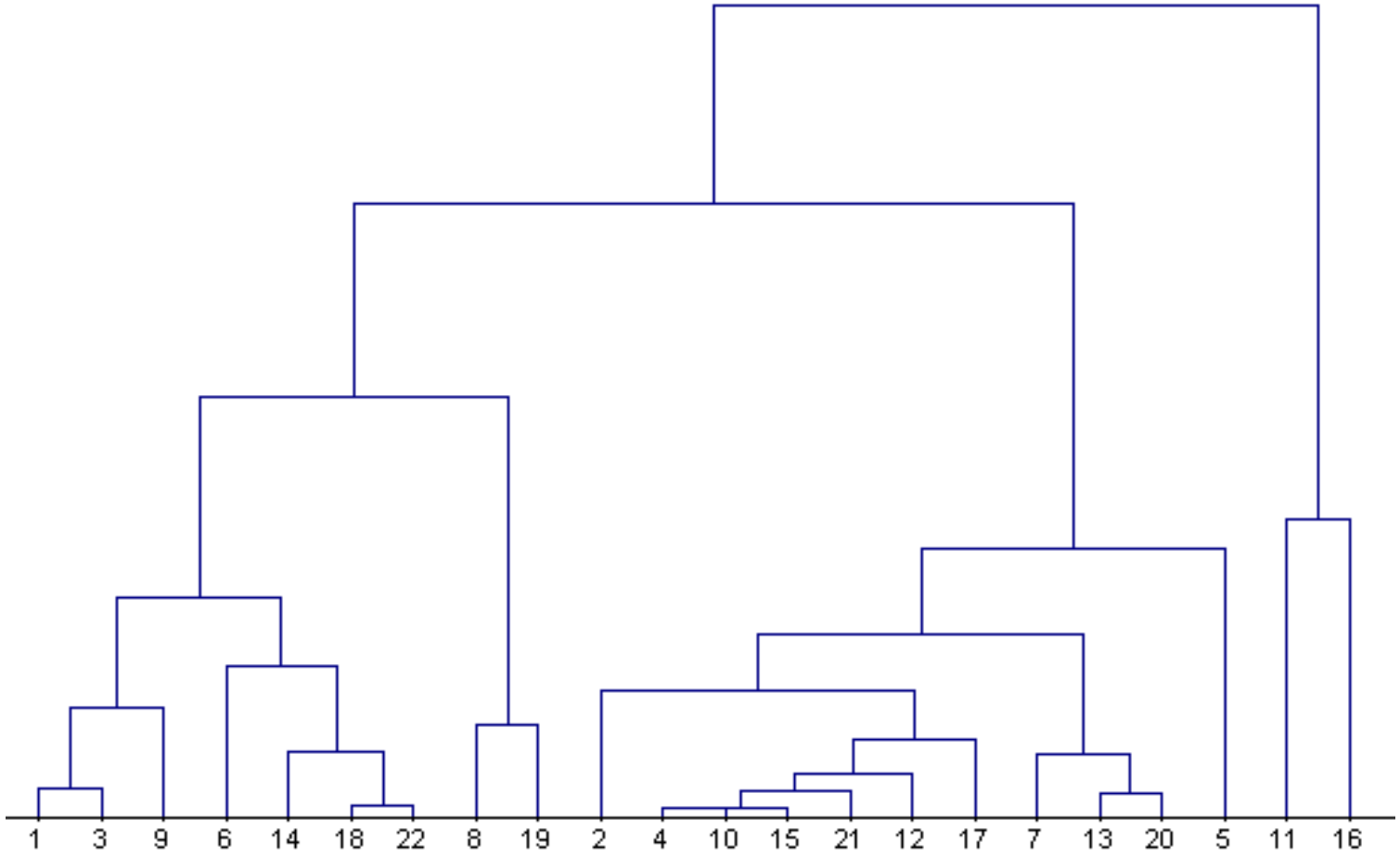
$$d(A,B) = \min d(a,b) \qquad d(A,B) = \max d(a,b)$$





Agglomerative clustering

Representation by a clustering tree (dend



How many clusters are there?

- Difficult to answer even for humans
- “Clustering tendency”, “cluster validity”
- Hierarchical methods:

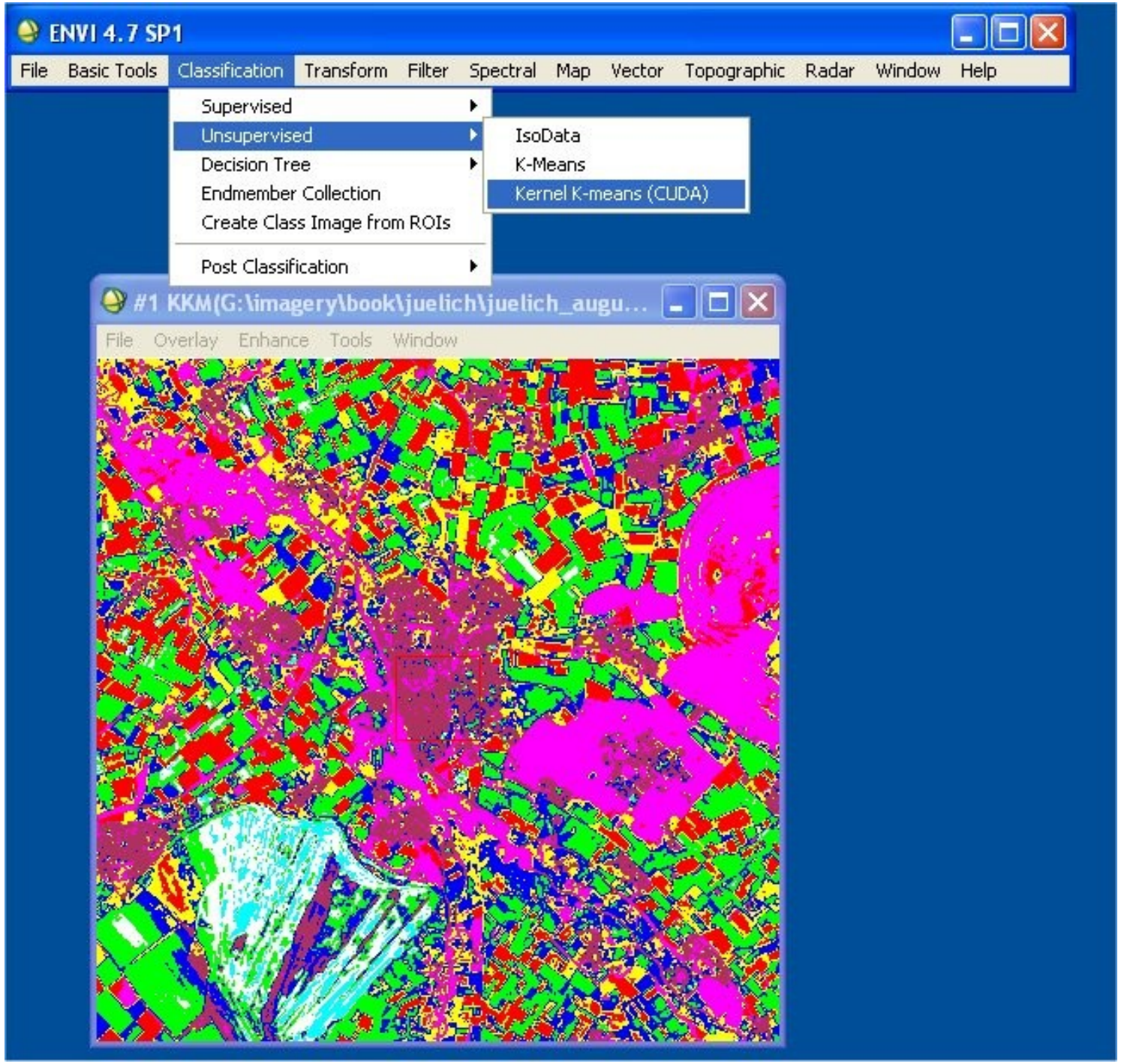
  N can be estimated from the complete dendrogram
- The methods minimizing a cost function

  N can be estimated from the “knees” in *J*-*N* graph

Applications of clustering in image proc.

- **Segmentation – clustering in color space**
- **Preliminary classification of multispectral images**
- **Clustering in parametric space – RANSAC, image registration and matching**

Numerous applications are outside image processing area


ENVI 4.7 SP1
File
Basic Tools
Classification
Transform
Filter
Spectral
Map
Vector
Topographic
Radar
Window
Help
Supervised
Unsupervised
Decision Tree
Endmember Collection
Create Class Image from ROIs
Post Classification
IsoData
K-Means
Kernel K-means (CUDA)
#1 KKM(G:\imagery\book\juelich\juelich_augu...
File
Overlay
Enhance
Tools
Window

Dimensionality reduction

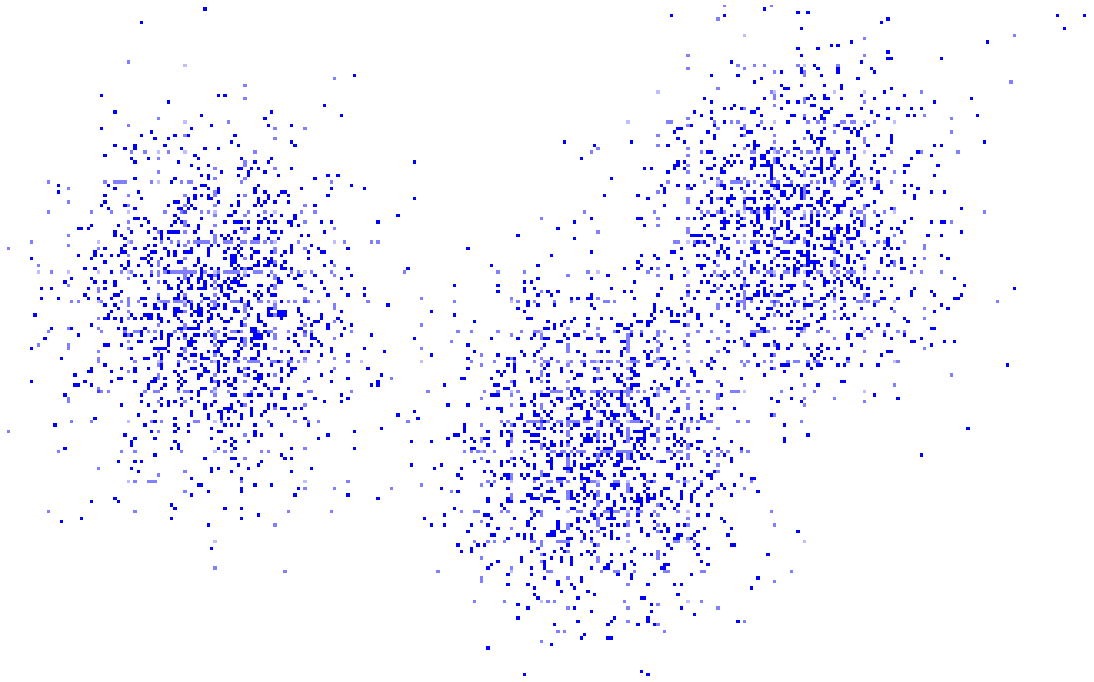
Having *D* features, we want to reduce their number to *n*, where $n \ll D$

$$(x_1, x_2, \cdots, x_D) \longrightarrow (y_1, y_2, \cdots, y_n)$$

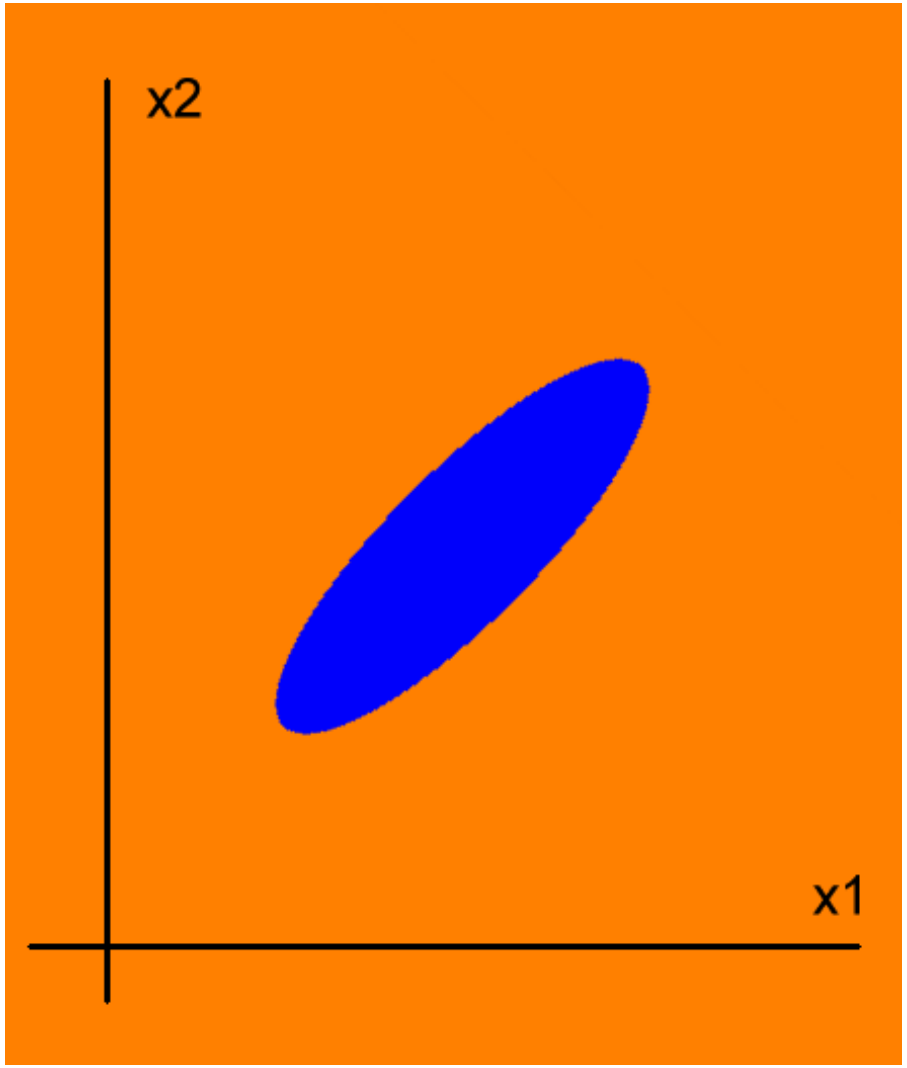
Principal Component Transform

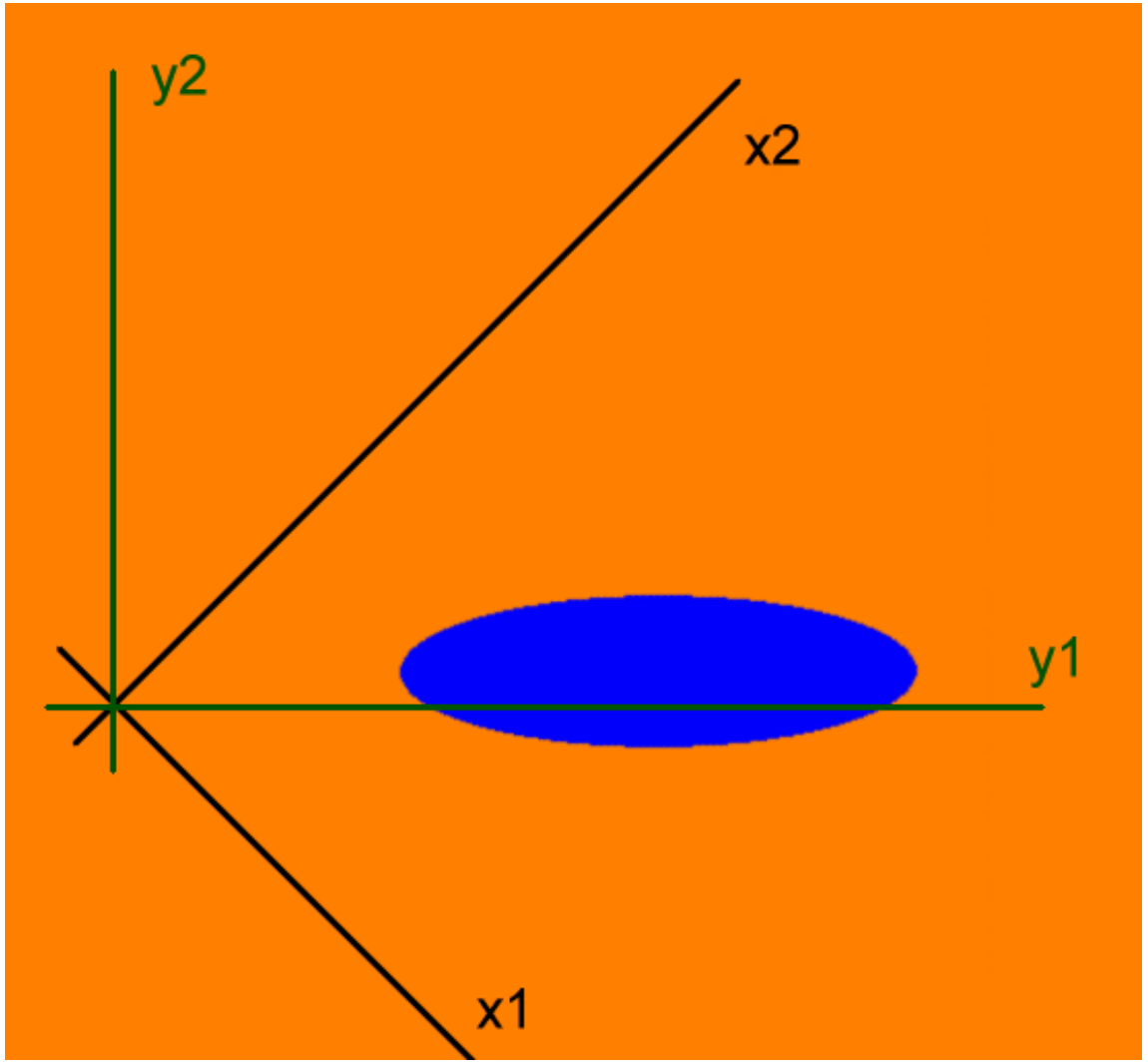
Karhunen-Loeve Transform

PCT is a method for “one-class” problem, i.e. for non-structured data (no class representatives, no training sets, just a cloud of data points in the feature space)



Principal Component Transform





Principal Component Transform

- PCT is a rotation of the feature space

$$y = T'x,$$

such that the new features y are **uncorrelated**, i.e. covariance matrix C_y is diagonal.

- This is always possible since the original covariance matrix C_x is symmetric and can be diagonalized in the orthonormal basis of its eigenvectors

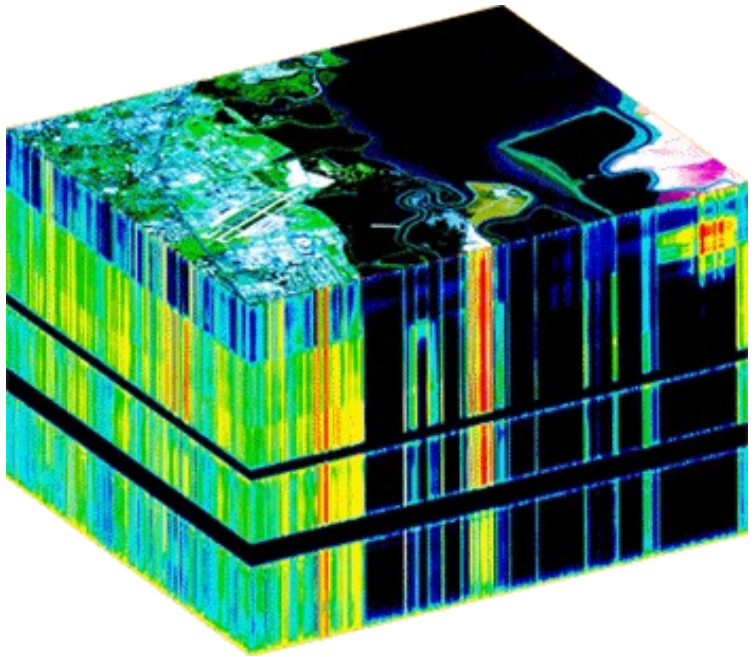
$$C_y = T'C_x T$$

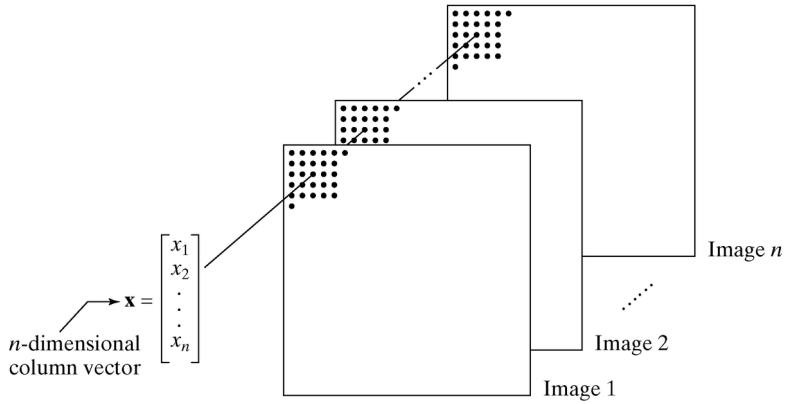
- Features with the highest variances are called **principal components.** First n PC's are kept.

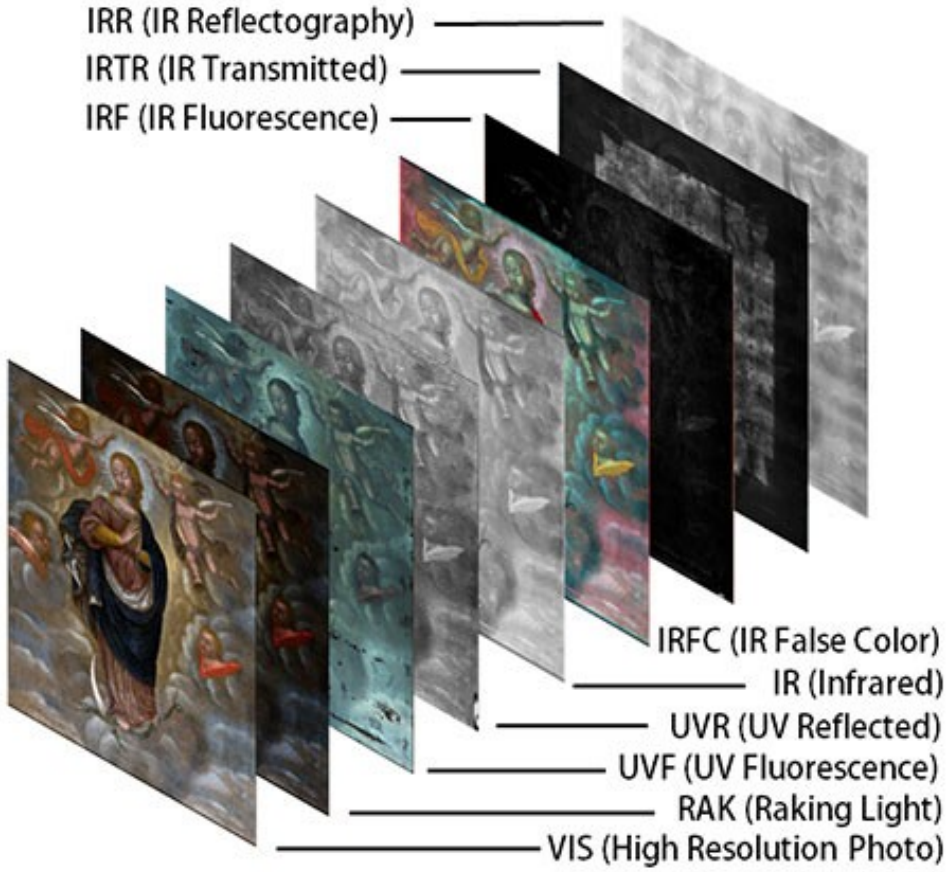
Applications of the PCT

- “Optimal” data representation
- Visualization and compression of multimodal images

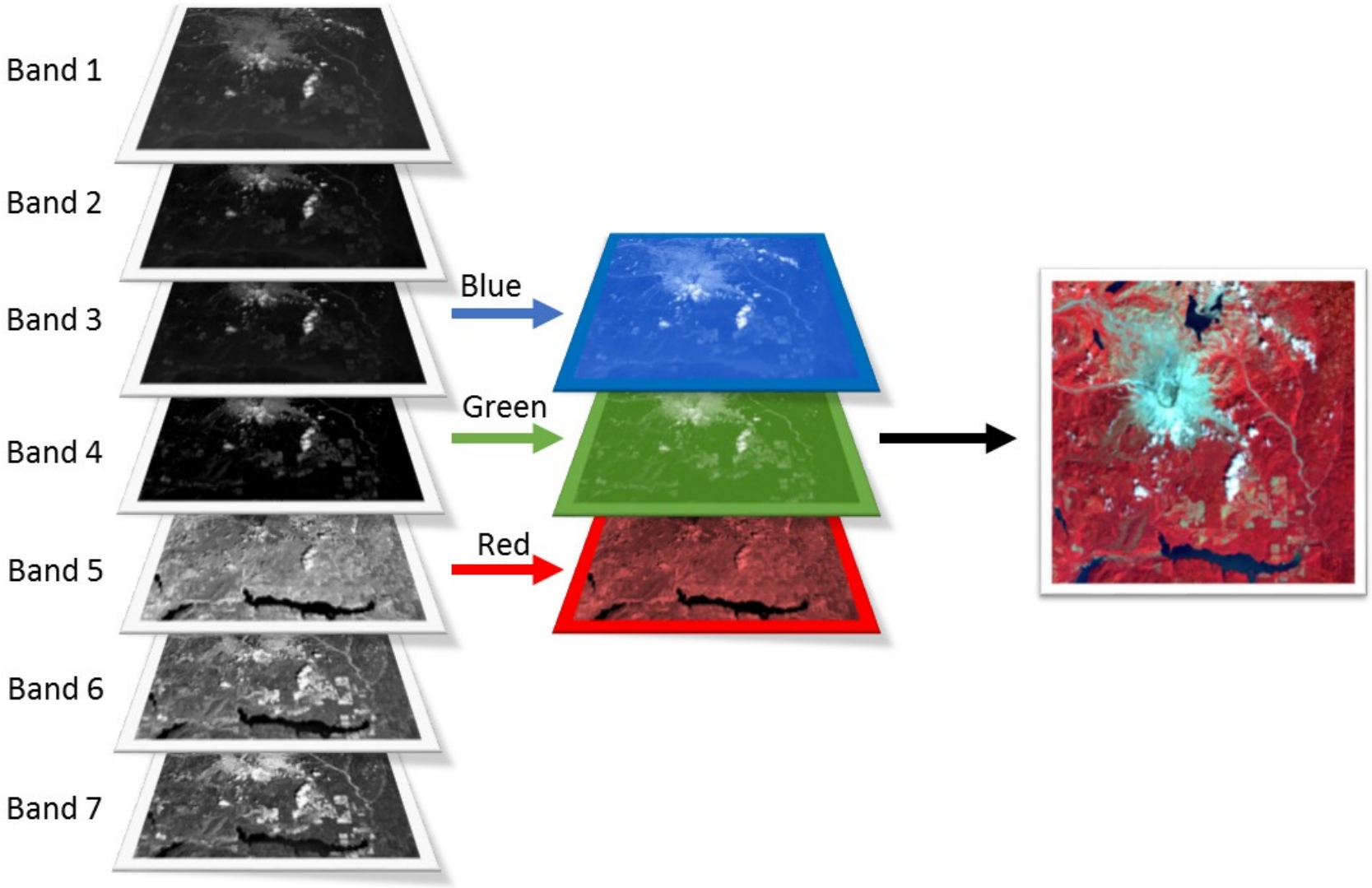
PCT of multispectral images



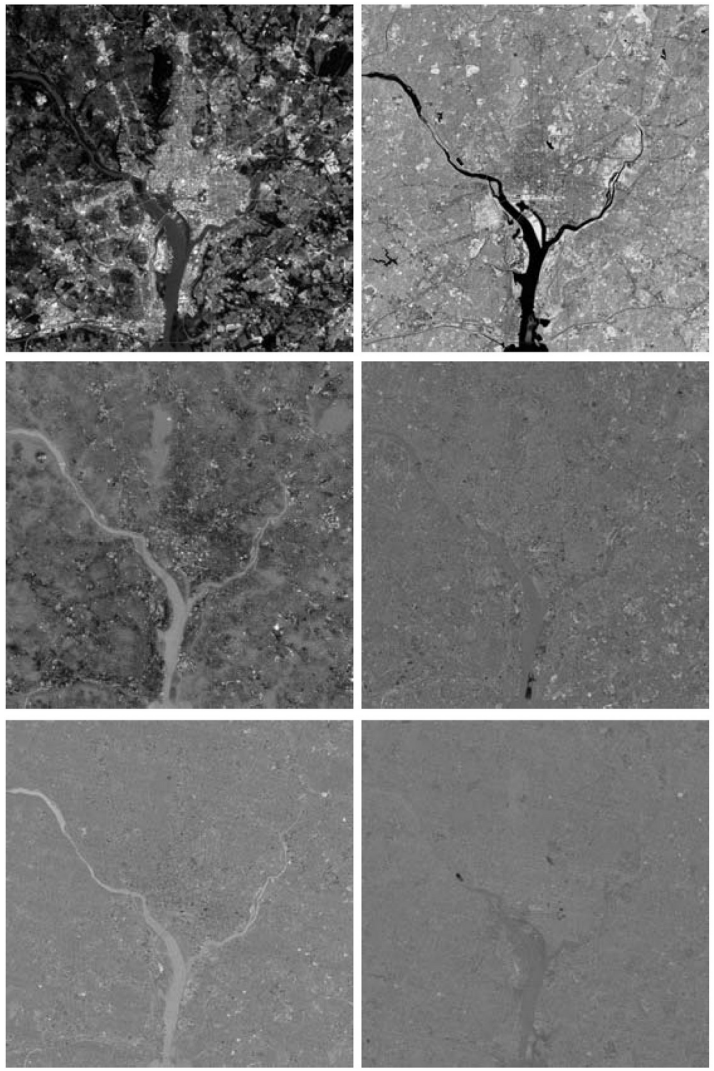




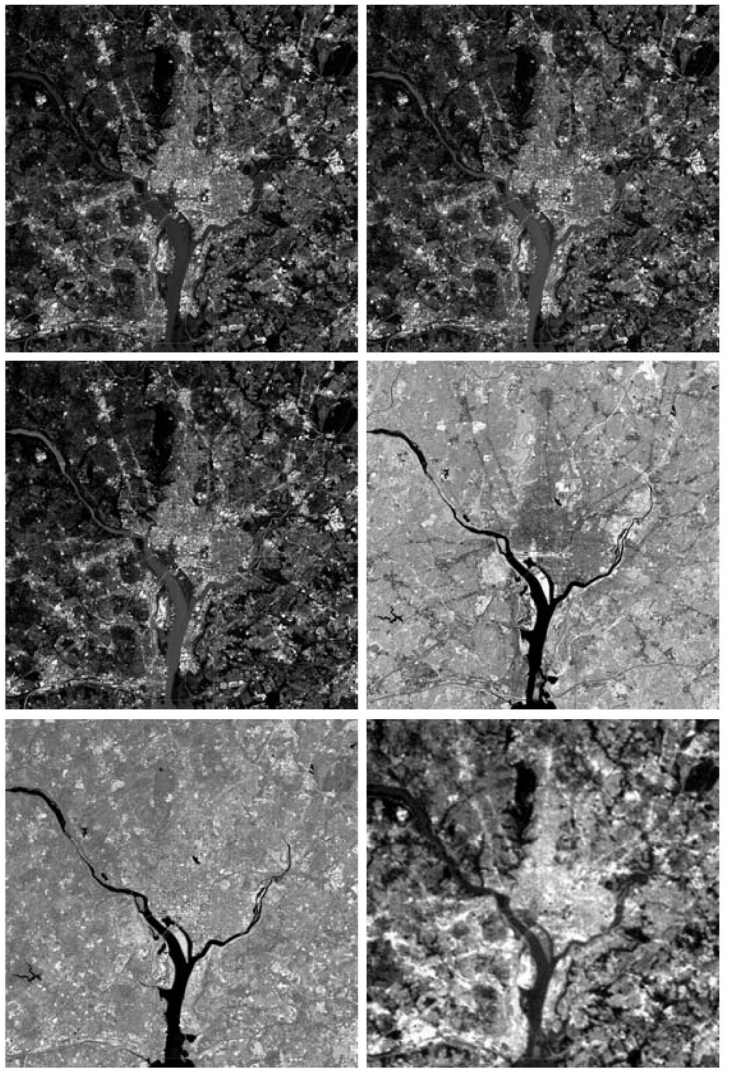
PCT for visualization



PCT



Reconstruction from two PC's



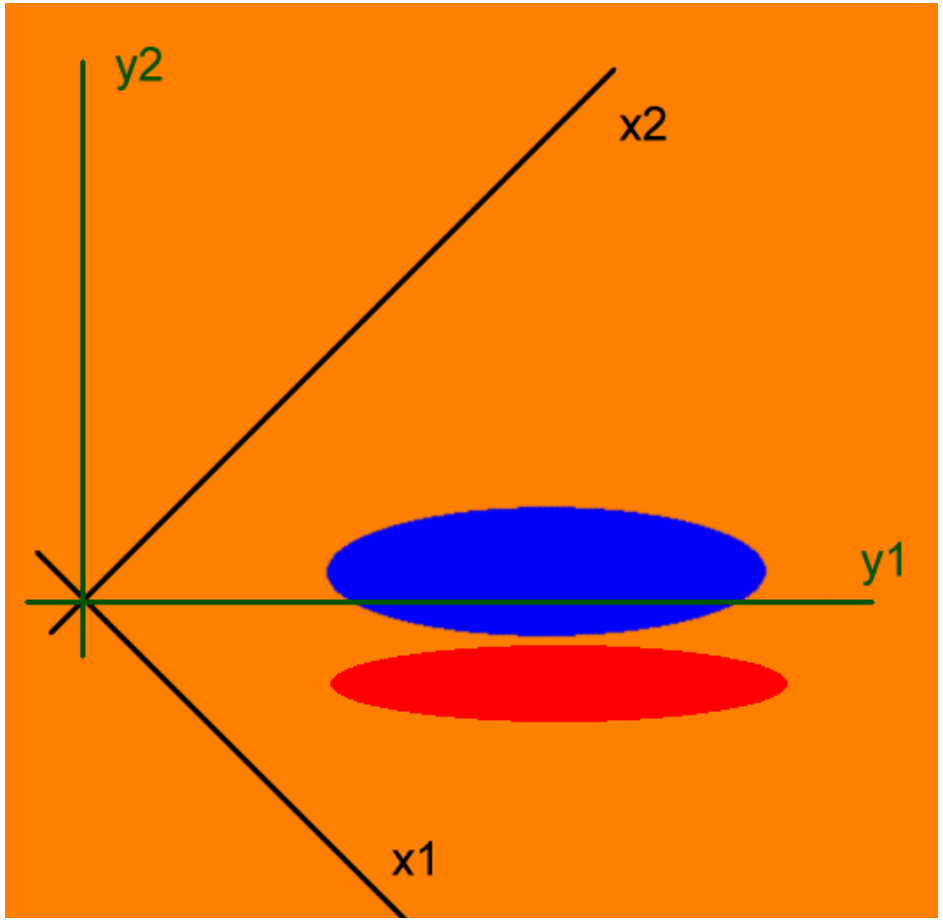
Original

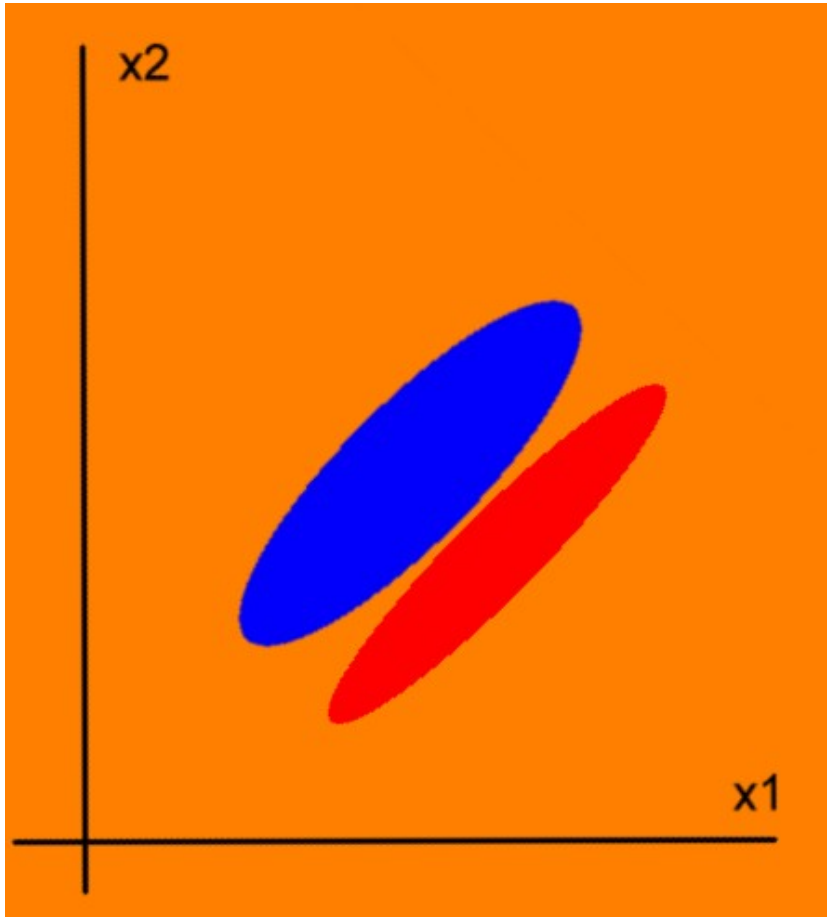
λ_1	λ_2	λ_3	λ_4	λ_5	λ_6
10352	2959	1403	203	94	31

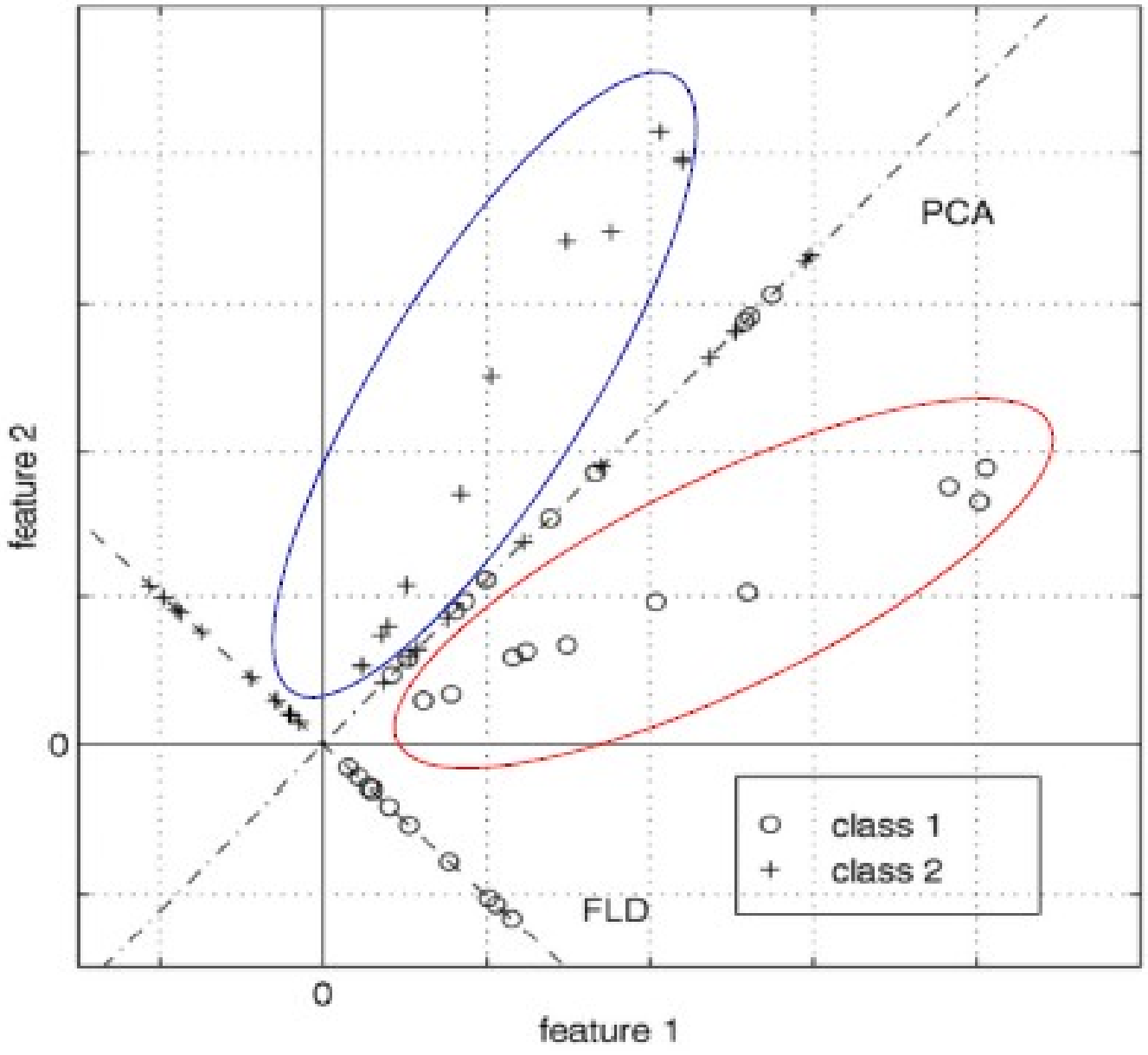
Why is PCT not suitable for classification?

PCT evaluates the contribution of individual features solely by their variances, which may be different from their **discrimination power**.

Why is PCT not suitable for classification?



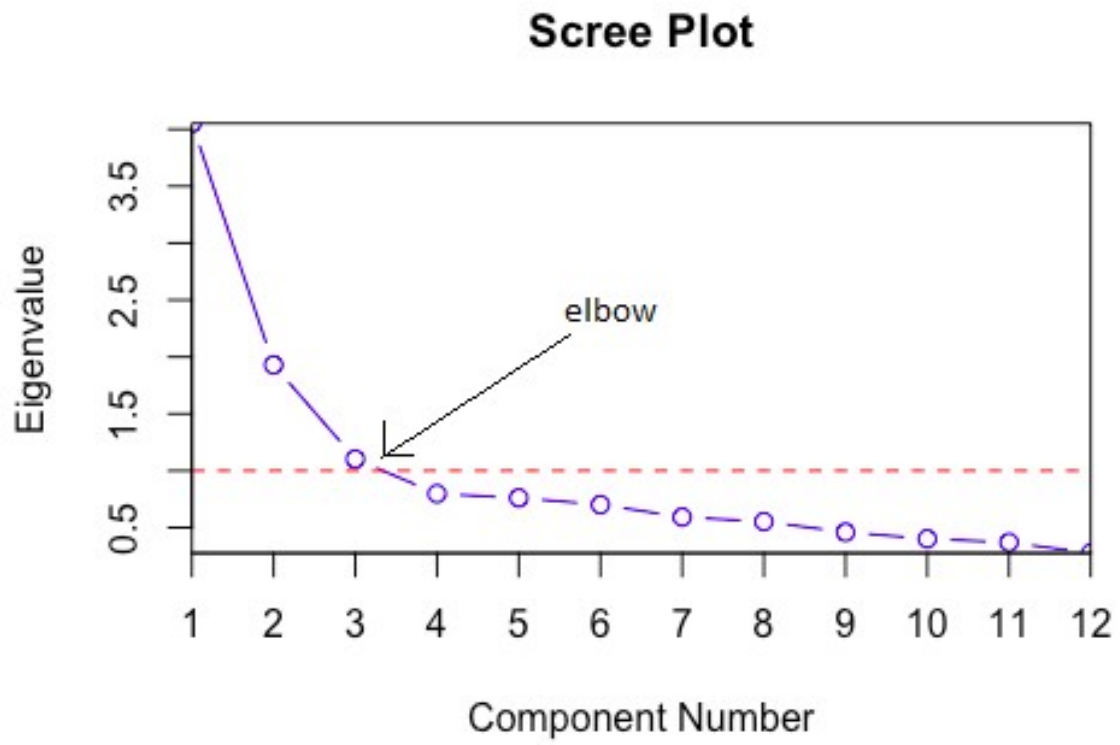



PCA
FLD
feature 2
feature 1
0
0
class 1
class 2

PCT: How many components?

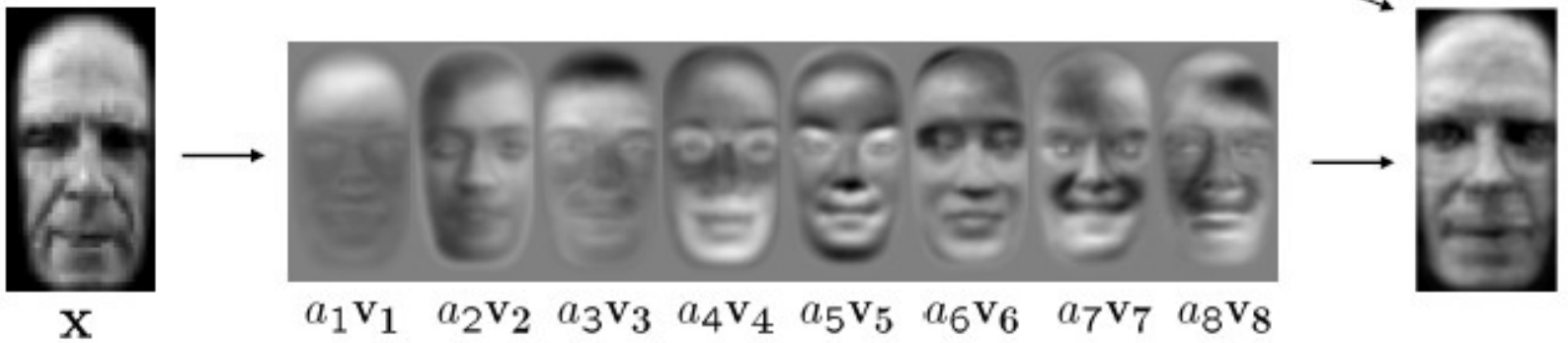
80% of total variation

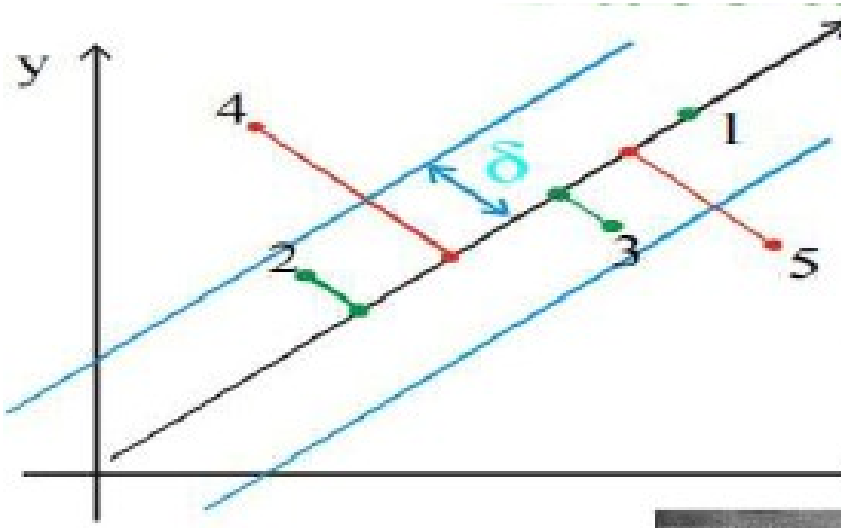
The “elbow” rule



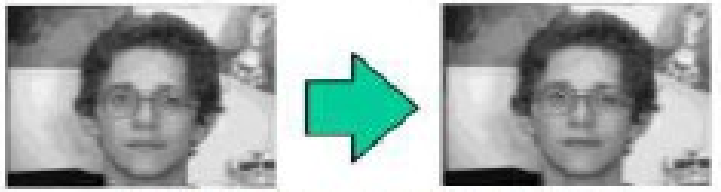
Face recognition by eigenfaces

$$\mathbf{x} \approx \overline{\mathbf{x}} + a_1\mathbf{v_1} + a_2\mathbf{v_2} + \ldots + a_K\mathbf{v_K}$$

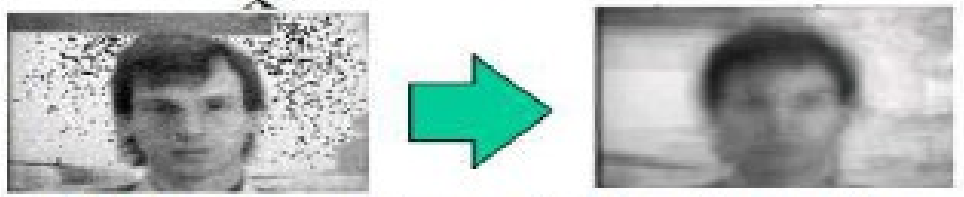




original projection



A face, used for training

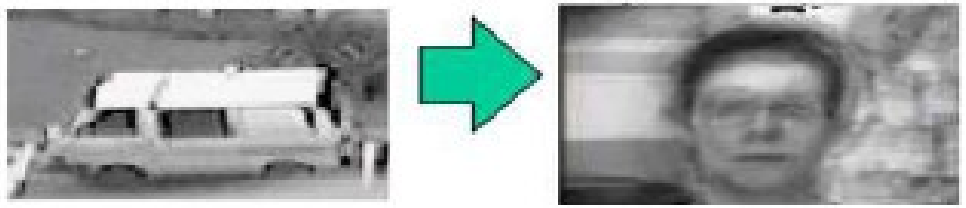


A face, not used for training

1 *The best case: on the subspace*

2,3 *Close enough*

4,5 *Too far – not a face*



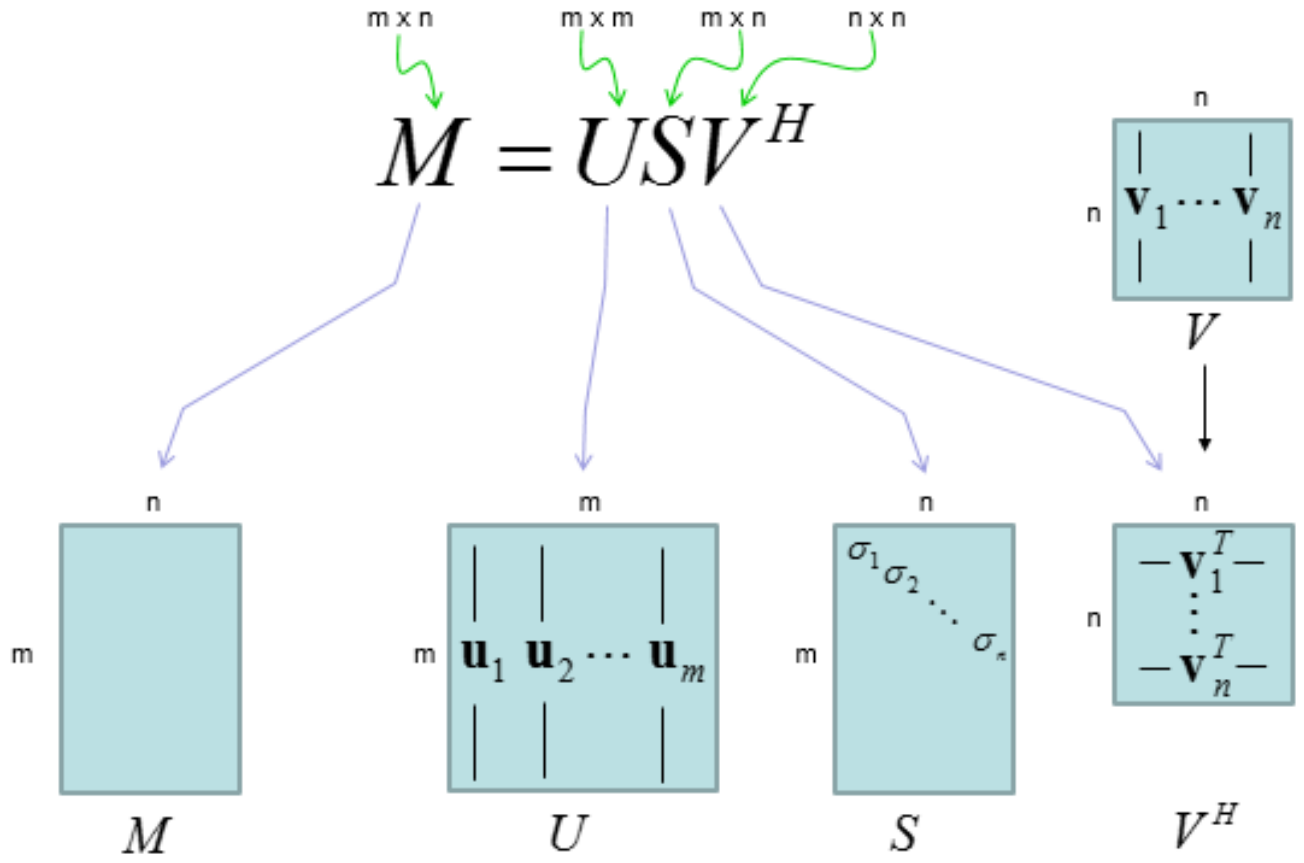
Not a face, not used for training

PCT vs. SVD

M – arbitrary matrix

U, V – orthogonal

S - diagonal

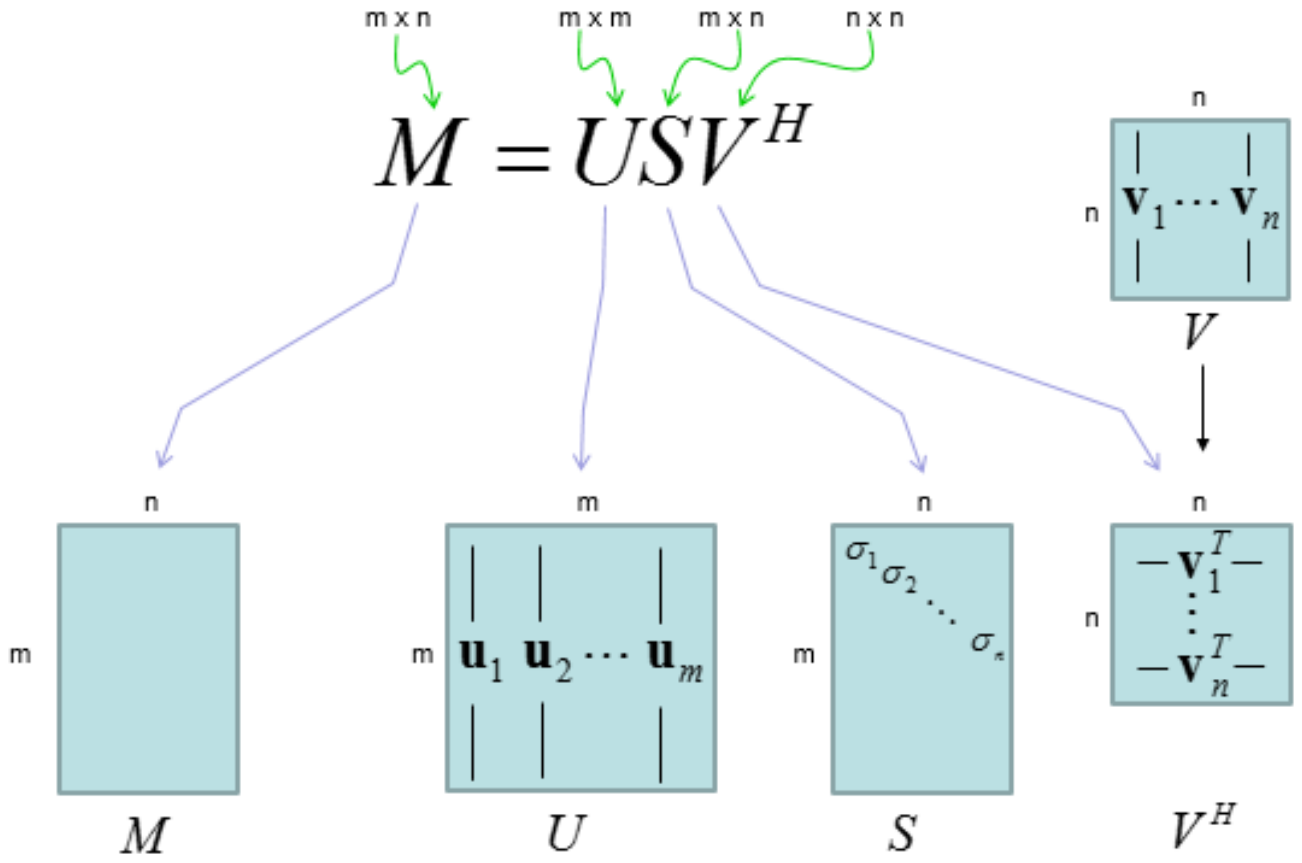


PCT vs. SVD

M – data matrix

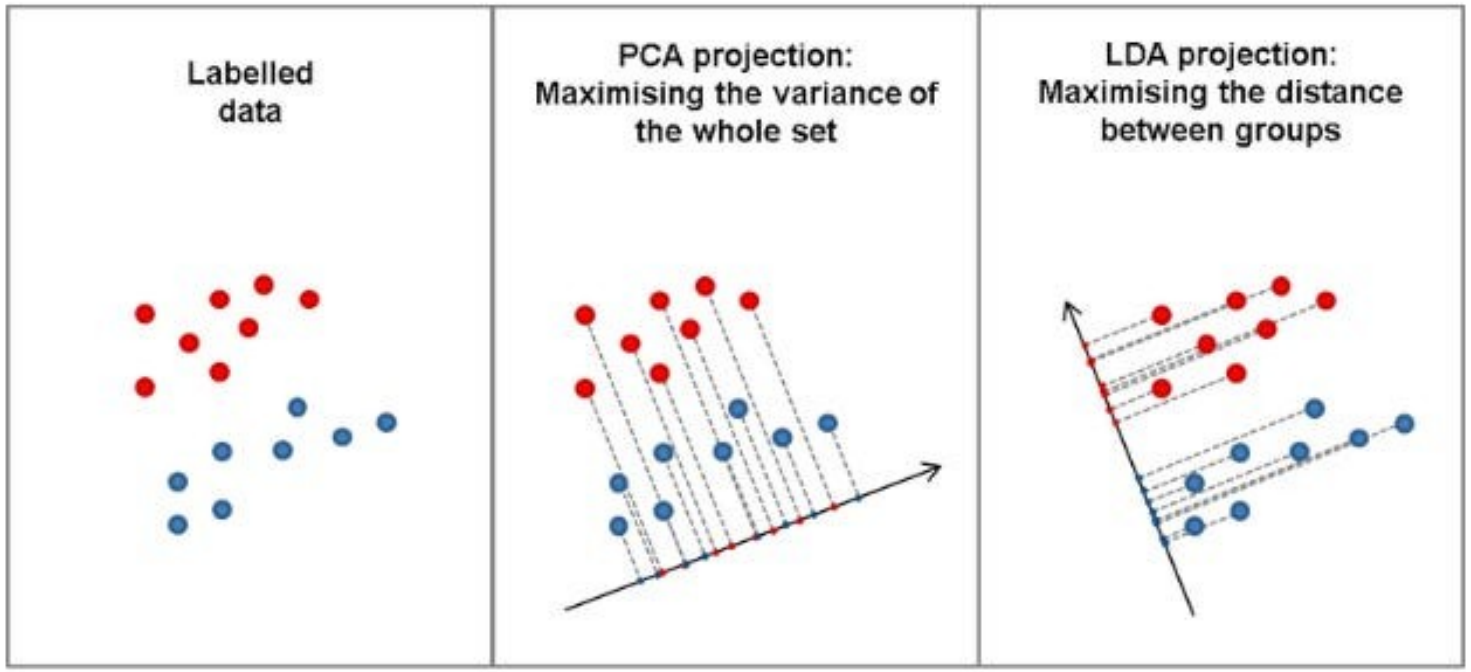
C = *M'M* – covariance matrix

SVD of *M* is the same as diagonalization of *C*



Linear discriminant analysis (LDA)

- Selection of the “best” feature/direction
- Optimal direction is given by the principal eigenvector of $(W^{-1}B)$



Thank you!

Any questions?